

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: supermarket_data = pd.read_csv('./supermarket_sales.csv')
```

```
In [3]: supermarket_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Invoice ID             1000 non-null   object
1   Branch                 1000 non-null   object
2   City                   1000 non-null   object
3   Customer type          1000 non-null   object
4   Gender                 1000 non-null   object
5   Product line           1000 non-null   object
6   Unit price             1000 non-null   float64
7   Quantity               1000 non-null   int64
8   Tax 5%                 1000 non-null   object
9   Total                  1000 non-null   float64
10  Date                   1000 non-null   object
11  Time                   1000 non-null   object
12  Payment                1000 non-null   object
13  cogs                   1000 non-null   float64
14  gross margin percentage 1000 non-null   float64
15  gross income           1000 non-null   float64
16  Rating                 1000 non-null   float64
dtypes: float64(6), int64(1), object(10)
memory usage: 132.9+ KB
```

Xử lý dữ liệu

Xử lý dữ liệu Tax bị sai kiểu dữ liệu

```
In [4]: supermarket_data_coppy = supermarket_data.copy()
supermarket_data_coppy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Invoice ID             1000 non-null   object
1   Branch                 1000 non-null   object
2   City                   1000 non-null   object
3   Customer type          1000 non-null   object
4   Gender                 1000 non-null   object
5   Product line           1000 non-null   object
6   Unit price             1000 non-null   float64
7   Quantity               1000 non-null   int64
8   Tax 5%                 1000 non-null   object
9   Total                  1000 non-null   float64
10  Date                   1000 non-null   object
11  Time                   1000 non-null   object
12  Payment                1000 non-null   object
13  cogs                   1000 non-null   float64
14  gross margin percentage 1000 non-null   float64
15  gross income           1000 non-null   float64
16  Rating                 1000 non-null   float64
dtypes: float64(6), int64(1), object(10)
memory usage: 132.9+ KB
```

- Công thức

- $\text{Total} = (\text{Unit Price} * \text{Quantity}) + \text{Tax}$

=> $\text{Tax} = \text{Total} - (\text{Unit Price} * \text{Quantity})$

```
In [5]: supermarket_data_copy['Tax'] = supermarket_data_copy['Total'] - (supermarket_data_copy['Unit price'] * supermarket_data_copy['Quantity'])
supermarket_data_copy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Invoice ID             1000 non-null   object
1   Branch                 1000 non-null   object
2   City                   1000 non-null   object
3   Customer type          1000 non-null   object
4   Gender                 1000 non-null   object
5   Product line           1000 non-null   object
6   Unit price             1000 non-null   float64
7   Quantity               1000 non-null   int64
8   Tax 5%                 1000 non-null   object
9   Total                  1000 non-null   float64
10  Date                   1000 non-null   object
11  Time                   1000 non-null   object
12  Payment                1000 non-null   object
13  cogs                   1000 non-null   float64
14  gross margin percentage 1000 non-null   float64
15  gross income           1000 non-null   float64
16  Rating                 1000 non-null   float64
17  Tax                    1000 non-null   float64
dtypes: float64(7), int64(1), object(10)
memory usage: 140.8+ KB
```

Xử lý dữ liệu bị sai kiểu dữ liệu thời gian

```
In [6]: supermarket_data_coppy['Date'] = pd.to_datetime(supermarket_data_coppy['Date'],
supermarket_data_coppy['Time'] = pd.to_datetime(supermarket_data['Time'], format
supermarket_data_coppy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Invoice ID             1000 non-null   object
1   Branch                 1000 non-null   object
2   City                   1000 non-null   object
3   Customer type          1000 non-null   object
4   Gender                 1000 non-null   object
5   Product line           1000 non-null   object
6   Unit price             1000 non-null   float64
7   Quantity               1000 non-null   int64
8   Tax 5%                 1000 non-null   object
9   Total                  1000 non-null   float64
10  Date                   1000 non-null   datetime64[ns]
11  Time                   1000 non-null   datetime64[ns]
12  Payment                1000 non-null   object
13  cogs                   1000 non-null   float64
14  gross margin percentage 1000 non-null   float64
15  gross income           1000 non-null   float64
16  Rating                 1000 non-null   float64
17  Tax                    1000 non-null   float64
dtypes: datetime64[ns](2), float64(7), int64(1), object(8)
memory usage: 140.8+ KB
```

Dataset này có bao nhiêu:

- Thành phố (City)?
- Chi nhánh (Branch)?
- Loại khách hàng (Customer type)?
- Dòng sản phẩm (Product line)?
- Hình thức thanh toán là gì ?

Có bao nhiêu chi nhánh

```
In [7]: branch_unique = supermarket_data_coppy['Branch'].unique()
print(f"Có {len(branch_unique)} Chi nhánh bao gồm {branch_unique}")
```

Có 3 Chi nhánh bao gồm ['A' 'C' 'B']

Có bao nhiêu thành phố

```
In [8]: city_unique = supermarket_data_coppy['City'].unique()
print(f"Có {len(city_unique)} thành phố bao gồm {city_unique}")
```

Có 3 thành phố bao gồm ['Yangon' 'Naypyitaw' 'Mandalay']

Có bao nhiêu loại khách hàng

```
In [9]: customer_type_unique = supermarket_data_coppy['Customer type'].unique()
print(f"Có {len(customer_type_unique)} loại khách hàng bao gồm {customer_type_un
```

Có 2 loại khách hàng bao gồm ['Member' 'Normal']

Có bao nhiêu dòng sản phẩm

```
In [10]: product_line_unique = supermarket_data_coppy['Product line'].unique()
print(f"Có {len(product_line_unique)} dòng sản phẩm bao gồm {product_line_unique
```

Có 6 dòng sản phẩm bao gồm ['Health and beauty' 'Electronic accessories' 'Home and lifestyle' 'Sports and travel' 'Food and beverages' 'Fashion accessories']

Có bao nhiêu hình thức thanh toán

```
In [11]: payment_unique = supermarket_data_coppy['Payment'].unique()
print(f"Có {len(payment_unique)} phương thức thanh toán bao gồm {payment_unique}
```

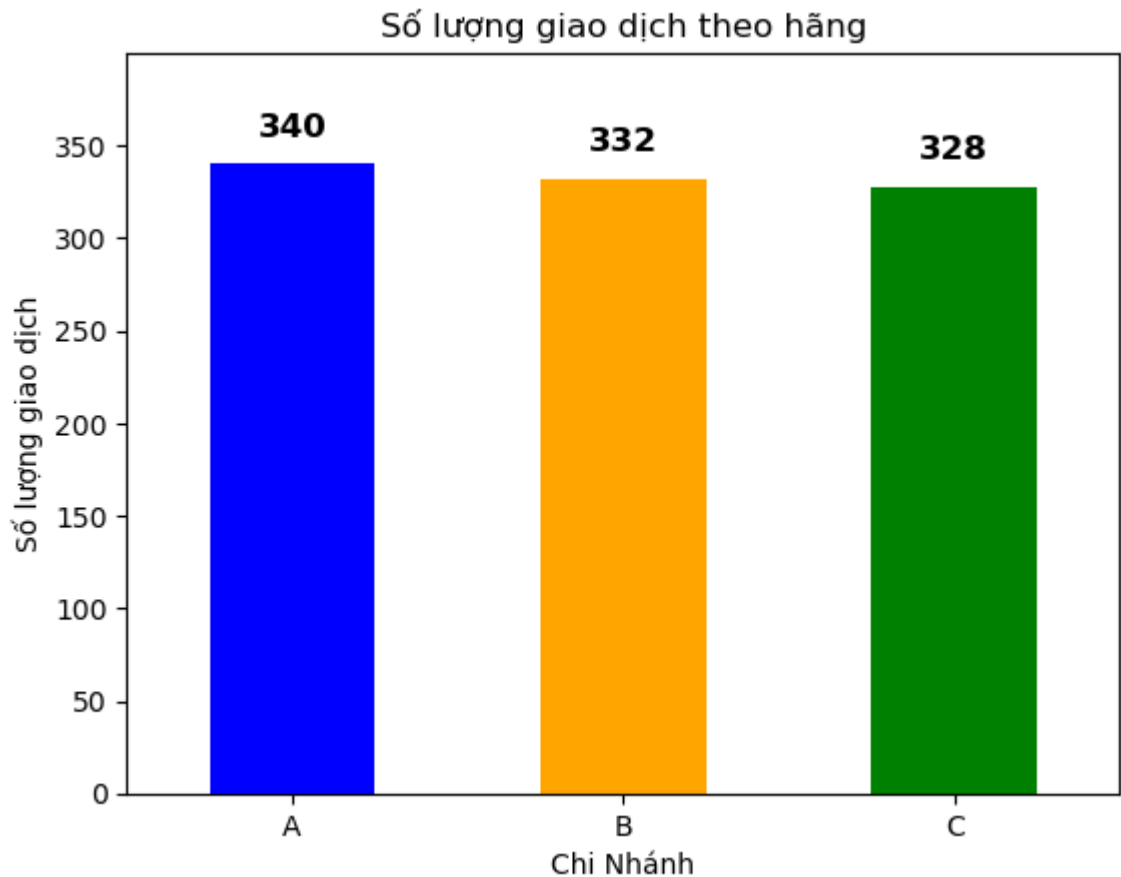
Có 3 phương thức thanh toán bao gồm ['Ewallet' 'Cash' 'Credit card']

Đếm các số lượng của các đặc dữ liệu

Đếm số lượng các giao dịch của các chi nhánh có trong dịch

```
In [12]: branch_counts = supermarket_data_coppy['Branch'].value_counts()

ax = branch_counts.plot(kind='bar', color=['blue', 'orange', 'green'])
ax.bar_label(ax.containers[0],
             labels=[f'{int(v)}' for v in branch_counts.values],
             padding=8, fontsize=12, fontweight='bold')
plt.xlabel('Chi Nhánh')
plt.ylabel('Số lượng giao dịch')
plt.title('Số lượng giao dịch theo hãng')
plt.ylim(0, ax.get_ylim()[1] * 1.12)
plt.xticks(ticks=range(len(branch_counts)), labels=branch_counts.index, rotation=45)
plt.show()
```



```
In [13]: percentage_branch = (branch_counts / branch_counts.sum()) * 100
for branch, percentage in percentage_branch.items():
    print(f'Tỷ lệ giao dịch của chi nhánh {branch}: {percentage:.2f}%')
```

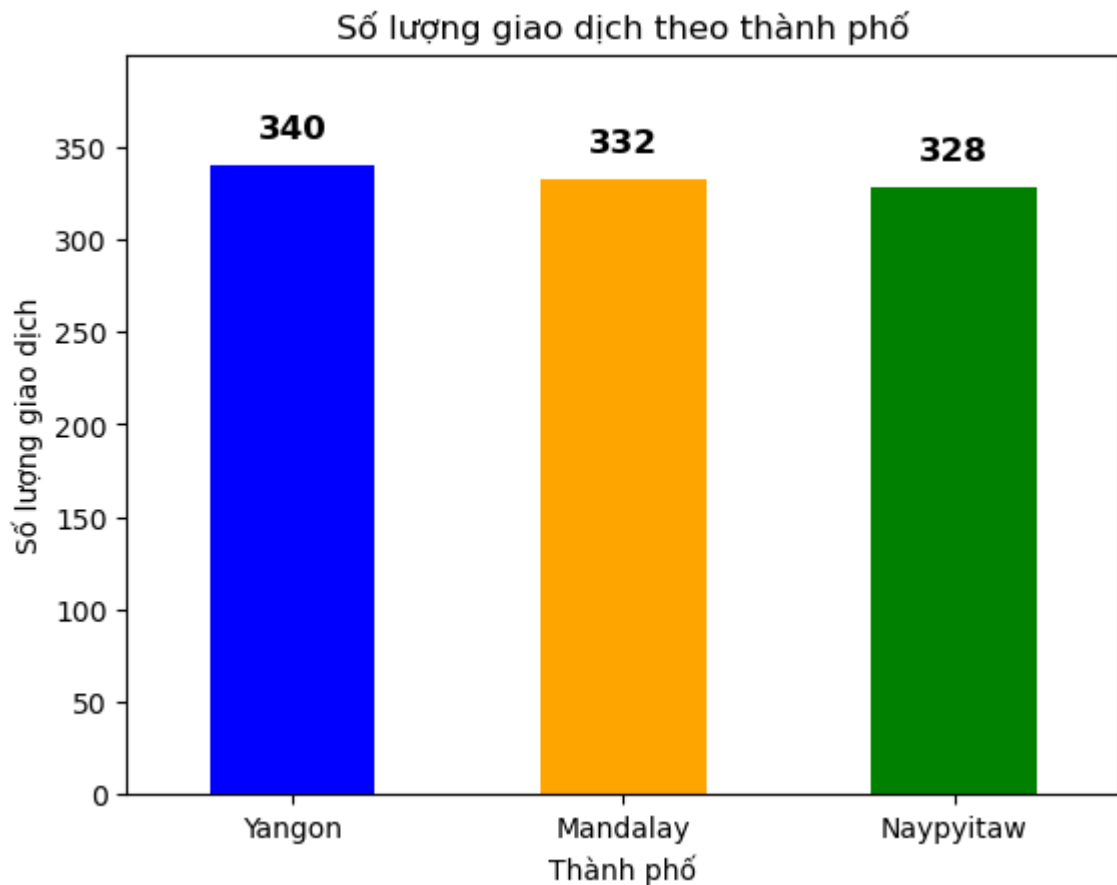
Tỷ lệ giao dịch của chi nhánh A: 34.00%
 Tỷ lệ giao dịch của chi nhánh B: 33.20%
 Tỷ lệ giao dịch của chi nhánh C: 32.80%

- Chi Nhánh A: Có 340 giao dịch
- Chi Nhánh B: Có 332 giao dịch
- Chi Nhánh C: Có 328 giao dịch

=> Không có sự chênh lệch quá lớn về các giao dịch của các hãng

Đếm số lượng giao dịch của các thành phố có trong dữ

```
In [14]: city_count = supermarket_data_coppy['City'].value_counts()
ax = city_count.plot(kind='bar', color=['blue', 'orange', 'green'])
ax.bar_label(ax.containers[0],
             labels=[f'{int(v)}' for v in city_count.values],
             padding=8, fontsize=12, fontweight='bold')
plt.xlabel('Thành phố')
plt.ylabel('Số lượng giao dịch')
plt.title('Số lượng giao dịch theo thành phố')
plt.ylim(0, ax.get_ylim()[1] * 1.12)
plt.xticks(ticks=range(len(city_count)), labels=city_count.index, rotation=0)
plt.show()
```



```
In [15]: # Tỷ lệ bán hàng giữa các thành phố
percentage_city = (city_count / city_count.sum()) * 100
for city, percentage in percentage_city.items():
    print(f'Tỷ lệ giao dịch của thành phố {city}: {percentage:.2f}%')
```

Tỷ lệ giao dịch của thành phố Yangon: 34.00%

Tỷ lệ giao dịch của thành phố Mandalay: 33.20%

Tỷ lệ giao dịch của thành phố Naypyitaw: 32.80%

- Kết luận:

- Thành phố Yangon có 340 giao dịch bằng với Chi Nhánh A
- Thành phố Mandalay có 332 giao dịch bằng với Chi Nhánh B
- Thành phố Napyitaw có 328 giao dịch bằng với Chi Nhánh C

=> không có sự chênh lệch quá lớn giữa các thành phố với giao dịch

=> Số lượng giao dịch giữa các thành phố và các Chi Nhánh bằng nhau

Tại sao số lượng giao dịch của các Chi Nhánh lại bằng nhau với các thành phố?

```
In [16]: # Xem mối quan hệ giữa Branch và City
print(pd.crosstab(supermarket_data_coppy['Branch'], supermarket_data_coppy['City
```

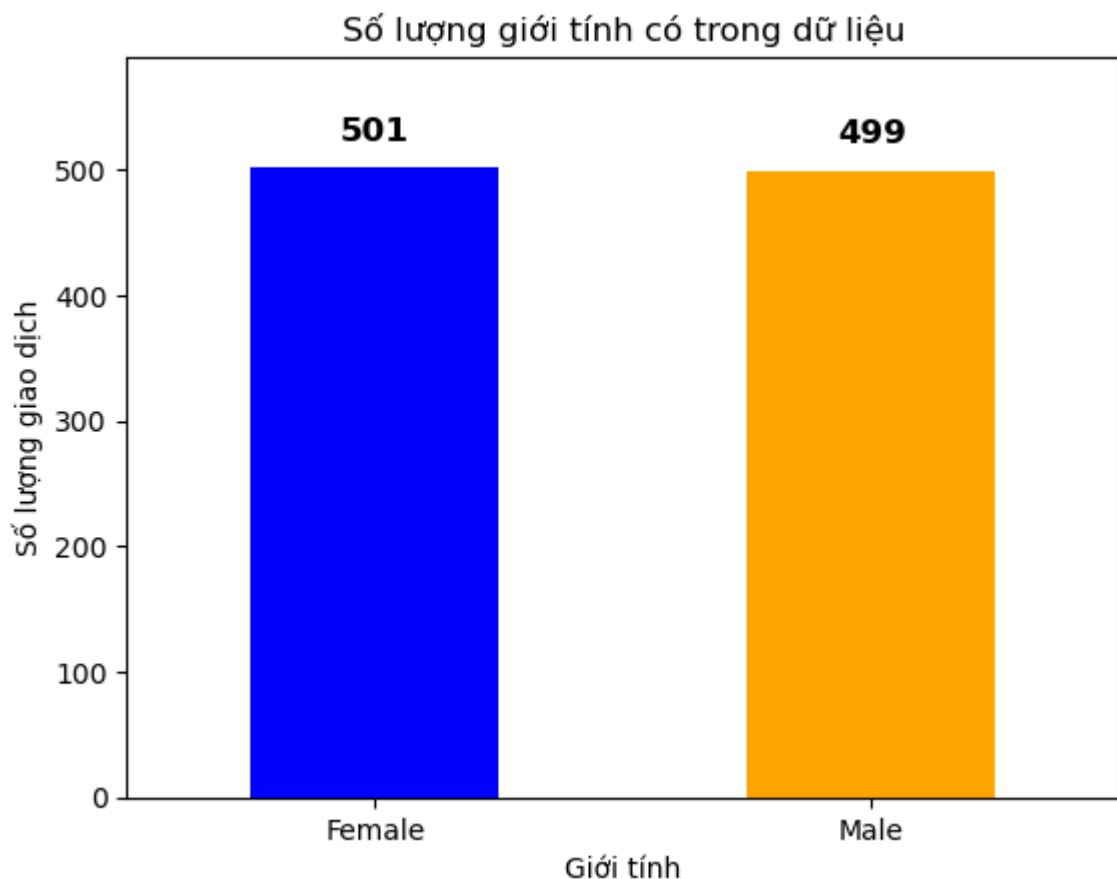
City	Mandalay	Naypyitaw	Yangon
Branch			
A	0	0	340
B	332	0	0
C	0	328	0

- Kết luận:

- Số lượng giao dịch giữa các thành phố và các Chi Nhánh giống nhau tại vì mỗi một Chi Nhánh bán ở một thành phố duy nhất

Đếm số lượng giao dịch theo giới tính

```
In [17]: gender_count = supermarket_data_copy['Gender'].value_counts()
ax = gender_count.plot(kind='bar', color=['blue', 'orange'])
ax.bar_label(ax.containers[0],
             labels=[f'{int(v)}' for v in gender_count.values],
             padding=8, fontsize=12, fontweight='bold')
plt.xlabel('Giới tính')
plt.ylabel('Số lượng giao dịch')
plt.title('Số lượng giới tính có trong dữ liệu')
plt.ylim(0, ax.get_ylim()[1] * 1.12)
plt.xticks(ticks=range(len(gender_count)), labels=gender_count.index, rotation=0)
plt.show()
```



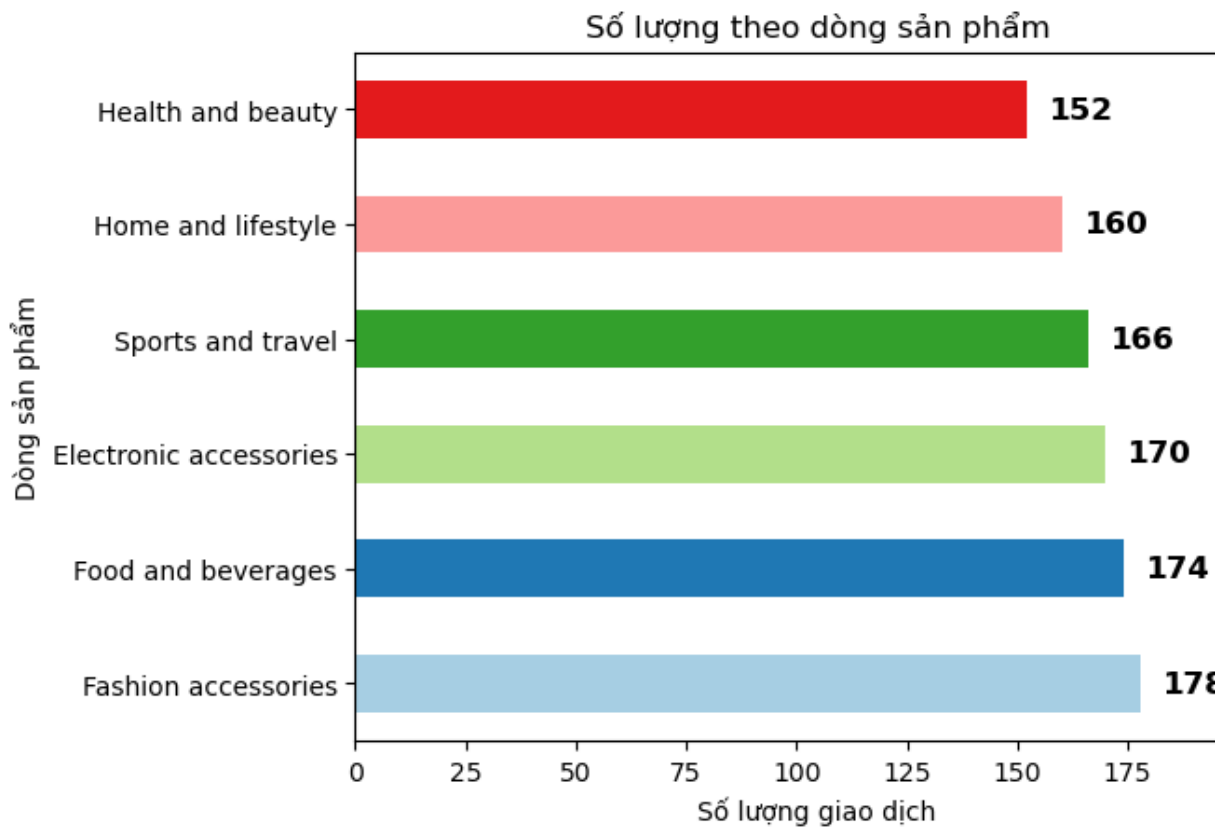
- Kết luận:

- Số lượng giao dịch giữa hai giới tính phân bố đều nhau không có sự chênh lệch đáng kể

Đếm số lượng sản phẩm có trong dữ liệu

```
In [18]: product_line_count = supermarket_data_copy['Product line'].value_counts()
ax = product_line_count.plot(kind='barh', color=plt.cm.Paired.colors)
```

```
ax.bar_label(ax.containers[0],
             labels=[f'{int(v)}' for v in product_line_count.values],
             padding=8, fontsize=12, fontweight='bold')
plt.xlabel('Số lượng giao dịch')
plt.ylabel('Dòng sản phẩm')
plt.title('Số lượng theo dòng sản phẩm')
plt.xlim(0, ax.get_xlim()[1] * 1.12)
plt.yticks(ticks=range(len(product_line_count)), labels=product_line_count.index)
plt.show()
```



```
In [19]: # Sự chênh lệch giữa các sản phẩm được bán ra
# Tổng số Lượng sản phẩm được bán ra theo từng dòng sản phẩm
product_line_count = supermarket_data_copy['Product line'].value_counts()
ten_sp = product_line_count.index.to_list()
sl = product_line_count.values.tolist()
print(sl)

# Sự chênh lệch giữa sản phẩm được bán nhiều nhất và ít nhất
chenh_lech = ((sl[0] - sl[-1])/sl[0]) * 100
print(f"Sự chênh lệch giữa sản phẩm được bán nhiều nhất và ít nhất là: {chenh_lech}")
```

[178, 174, 170, 166, 160, 152]

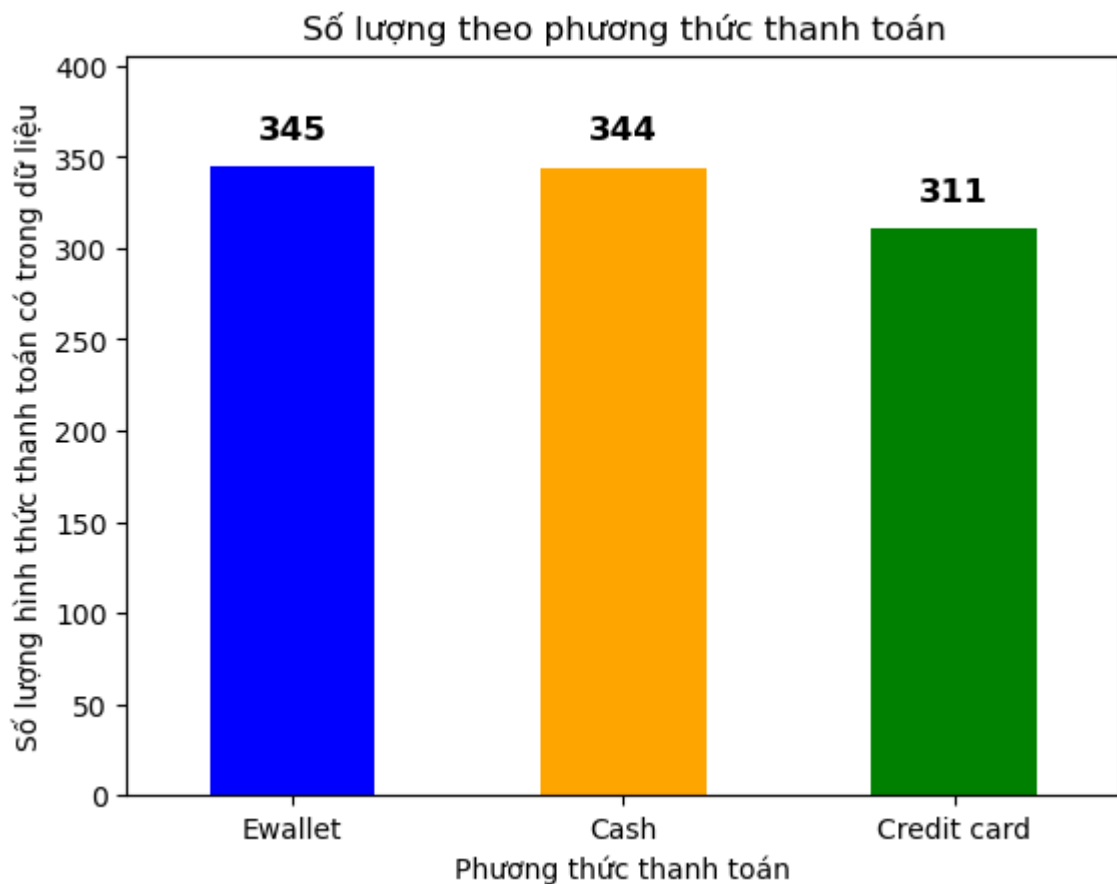
Sự chênh lệch giữa sản phẩm được bán nhiều nhất và ít nhất là: 14.606741573033707 sản

- Kết luận:

- Không có sự chênh lệch quá lớn giữa các sản phẩm
- Các sản phẩm dẫn đầu là: Fashion accessories, Food and beverages, Electric accessories
- Sự chênh lệch giữa sản phẩm bán cao nhất và thấp nhất không đáng kể chỉ từ: 14.6067%

Đếm số lượng theo hình thức thanh toán trong dữ liệu

```
In [20]: payment_count = supermarket_data_coppy['Payment'].value_counts()
ax = payment_count.plot(kind='bar', color=['blue', 'orange', 'green'])
ax.bar_label(ax.containers[0],
             labels=[f'{int(v)}' for v in payment_count.values],
             padding=8, fontsize=12, fontweight='bold')
plt.xlabel('Phương thức thanh toán')
plt.ylabel('Số lượng hình thức thanh toán có trong dữ liệu')
plt.title('Số lượng theo phương thức thanh toán')
plt.ylim(0, ax.get_ylim()[1] * 1.12)
plt.xticks(ticks=range(len(payment_count)), labels=payment_count.index, rotation=45)
plt.show()
```



- Kết luận:

- Siêu thị này các hình thức thanh toán phân bố đồng đều không có sự chênh lệch đáng kể

Số lượng giao dịch theo thời gian

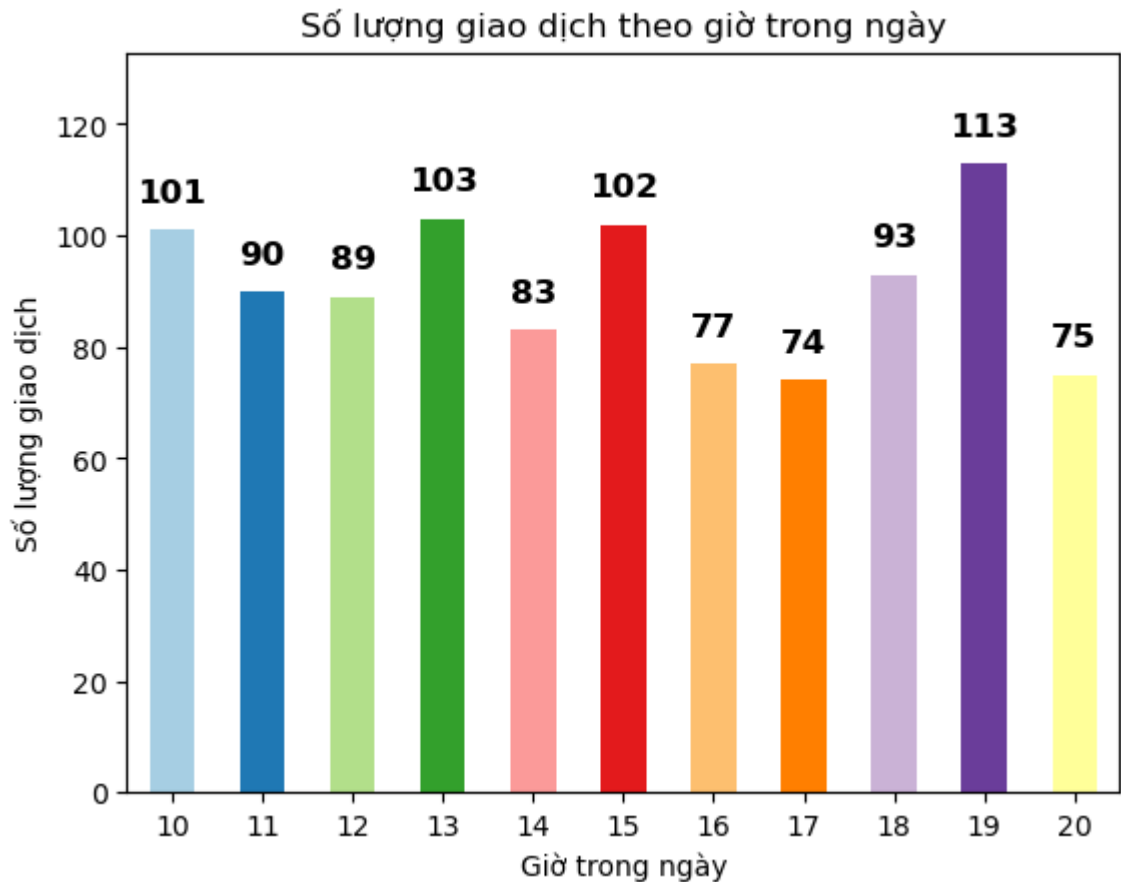
Số lượng giao dịch theo giờ trong ngày

```
In [21]: # Vẽ biểu đồ mua hàng theo thời gian trong ngày
supermarket_data_coppy['Hour'] = supermarket_data_coppy['Time'].dt.hour
hourly_counts = supermarket_data_coppy['Hour'].value_counts().sort_index()
ax = hourly_counts.plot(kind='bar', color=plt.cm.Paired.colors)
ax.bar_label(ax.containers[0],
```

```

labels=[f'{int(v)}' for v in hourly_counts.values],
padding=8, fontsize=12, fontweight='bold')
plt.xlabel('Giờ trong ngày')
plt.ylabel('Số lượng giao dịch')
plt.title('Số lượng giao dịch theo giờ trong ngày')
plt.ylim(0, ax.get_ylim()[1] * 1.12)
plt.xticks(ticks=range(len(hourly_counts)), labels=hourly_counts.index, rotation
plt.show()

```



- Phân tích:

- 19h là khoảng thời gian có nhiều khách hàng mua hàng nhất trong ngày, nguyên nhân đây là khoảng thời gian khách hàng rảnh nhất trong ngày nên họ dành thời gian để đi mua sắm
- 17h là khoảng thời gian khách hàng mua hàng ít nhất, nguyên nhân có thể khoảng thời gian khách hàng đang làm việc

Số lượng giao dịch theo các buổi trong ngày

```

In [22]: def get_time_of_day(hour):
        if pd.isna(hour):
            return 'Không xác định' # hoặc drop những hàng này nếu không muốn hiển
        hour = int(hour) # đảm bảo là số nguyên
        if 6 <= hour < 11:
            return 'Sáng' # Buổi sáng: 6h - 10h59
        elif 11 <= hour < 14:
            return 'Trưa' # Buổi trưa: 11h - 13h59
        elif 14 <= hour < 18:
            return 'Chiều' # Buổi chiều: 14h - 17h59

```

```

elif 18 <= hour < 24:
    return 'Tối'          # Buổi tối: 18h - 23h59

# Tạo cột Time of Day (chạy sau khi làm sạch Hour)
supermarket_data_coppy['Time of Day'] = supermarket_data_coppy['Hour'].apply(get)

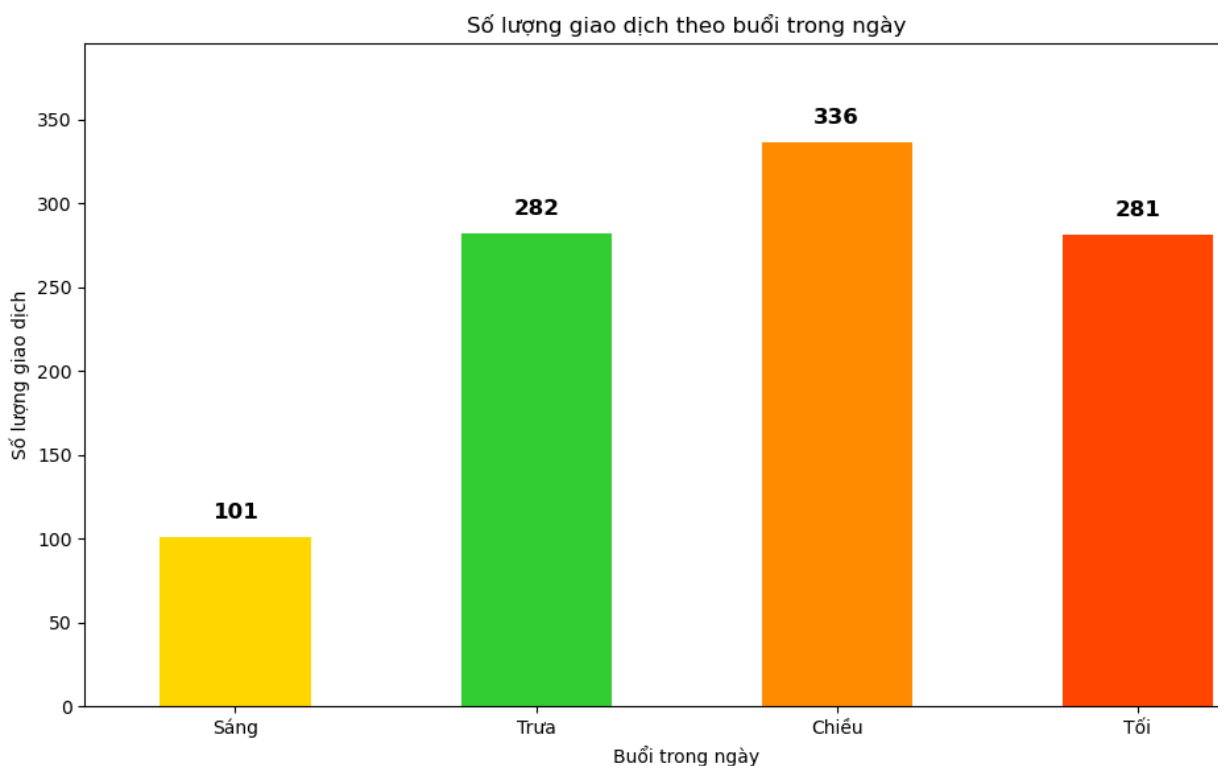
# Đếm và sắp xếp thứ tự mong muốn
order = ['Sáng', 'Trưa', 'Chiều', 'Tối']
time_of_day_counts = supermarket_data_coppy['Time of Day'].value_counts().reindex(order)
# Vẽ biểu đồ
colors = ['#FFD700', '#32CD32', '#FF8C00', '#FF4500'] # Màu cho từng buổi + Khác

ax = time_of_day_counts.plot(
    kind='bar',
    color=colors,
    figsize=(10, 6) # Tăng kích thước để hiển thị 4-6 cột rõ hơn
)

# Thêm nhãn số lên đầu cột (an toàn với NaN)
ax.bar_label(
    ax.containers[0],
    labels=[f'{int(v)}' if not pd.isna(v) else '0' for v in time_of_day_counts.values],
    padding=8,
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Buổi trong ngày')
plt.ylabel('Số lượng giao dịch')
plt.title('Số lượng giao dịch theo buổi trong ngày')
plt.ylim(0, ax.get_ylim()[1] * 1.12)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

```



- Phân tích:

- Mặc dù 19h là khoảng thời gian khách hàng mua nhiều nhất nhưng buổi chiều thời gian khác -18h lại là buổi có số lượng khách hàng mua sắm nhiều nhất vì đây là khoảng thời gian khách tan làm có nhiều nhu cầu mua hàng hơn
- Thời gian này có nhiều lý do để họ mua hàng có thể là chuẩn bị bữa tối, giải trí sau giờ làm,

Đếm số lượng giao dịch theo từng tháng

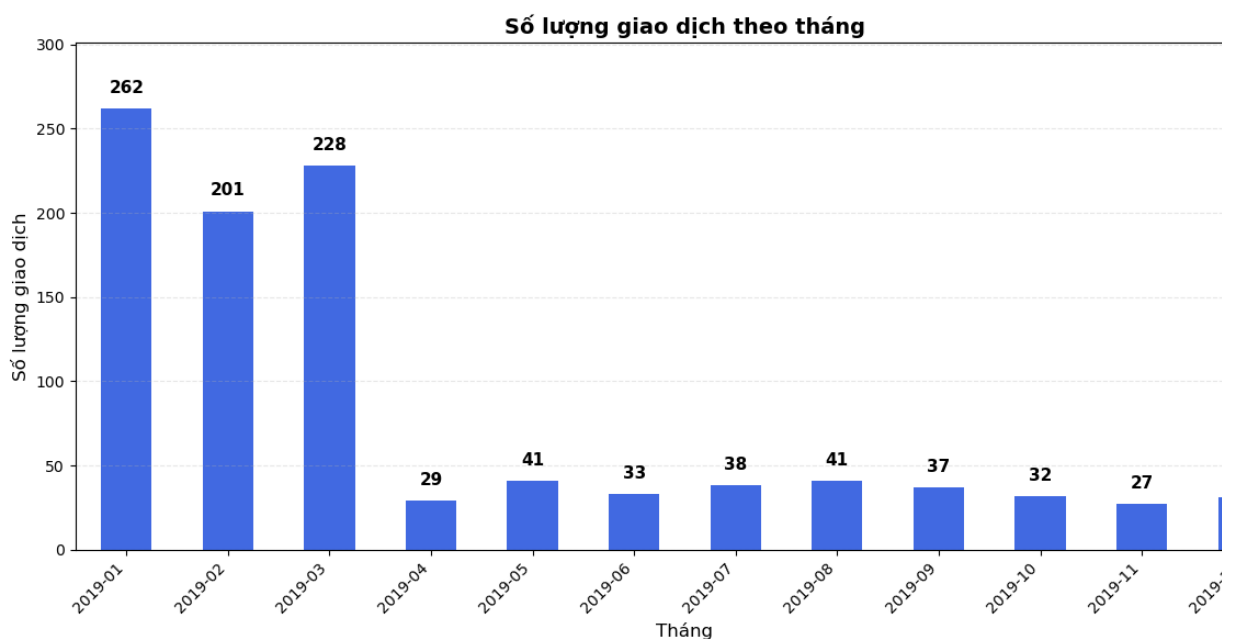
```
In [23]: # Giả sử supermarket_data đã có cột 'Date' dạng datetime
supermarket_data_copy['YearMonth'] = supermarket_data_copy['Date'].dt.to_period('M')

# Đếm số giao dịch theo tháng
monthly_counts = supermarket_data_copy.groupby('YearMonth').size()

# Vẽ biểu đồ
plt.figure(figsize=(12, 6))
ax = monthly_counts.plot(kind='bar', color='royalblue')

# Thêm số lên đầu cột
ax.bar_label(ax.containers[0],
             labels=[f'{int(v)}' for v in monthly_counts.values],
             padding=8, fontsize=11, fontweight='bold')

plt.title('Số lượng giao dịch theo tháng', fontsize=14, fontweight='bold')
plt.xlabel('Tháng', fontsize=12)
plt.ylabel('Số lượng giao dịch', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.ylim(0, monthly_counts.max() * 1.15)
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```



```
In [24]: # Sự chênh lệch giữa tháng 1 và tháng 12
jan_count = monthly_counts.get('2019-01', 0)
```

```
dec_count = monthly_counts.get('2019-12', 0)
chenh_lech_jan_dec = ((dec_count - jan_count) / jan_count) * 100 if jan_count != 0 else 0
print(f"Sự chênh lệch giữa tháng 1 và tháng 12 là: {chenh_lech_jan_dec} %")
```

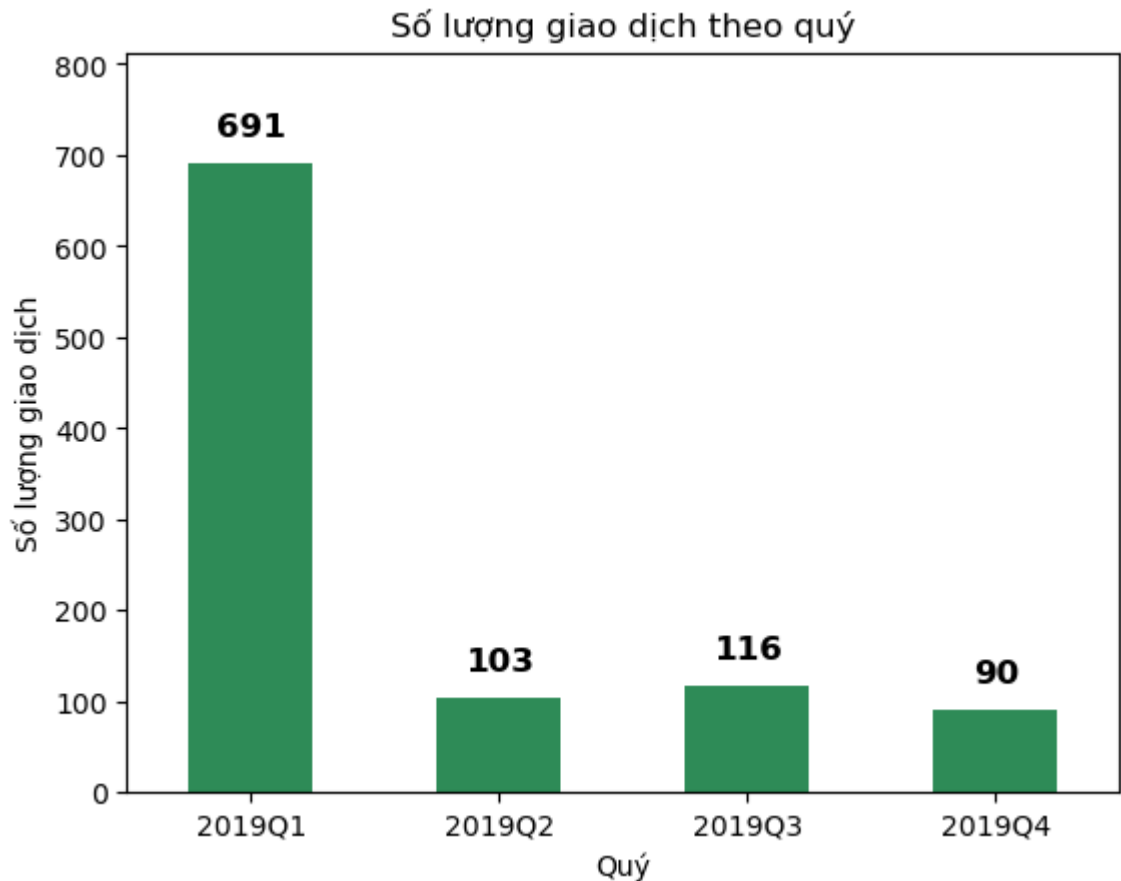
Sự chênh lệch giữa tháng 1 và tháng 12 là: -88.16793893129771 %

- Kết luận:

- Các tháng đầu năm khách hàng có xu hướng mua hàng nhiều năm hơn
- Sau đó giảm mạnh vào quý II, III và IV
- Trong tháng 1 khách hàng có xu hướng mua sắm nhiều hơn sau đó giảm nhẹ vào tháng 2 và tháng 3 nhưng các tháng tiếp theo thì giảm mạnh
- Sự chênh lệch giữa tháng 1 và tháng 12 là 88.12% cho thấy có sự chênh lệch lớn giữa tháng đầu năm và cuối năm

Đếm số lượng giao dịch theo quý

```
In [25]: # đếm số lượng giao dịch theo quý
supermarket_data_coppy['Quarter'] = supermarket_data_coppy['Date'].dt.to_period('Q')
quarterly_counts = supermarket_data_coppy.groupby('Quarter').size()
ax = quarterly_counts.plot(kind='bar', color='seagreen')
ax.bar_label(ax.containers[0],
             labels=[f'{int(v)}' for v in quarterly_counts.values],
             padding=8, fontsize=12, fontweight='bold')
plt.xlabel('Quý')
plt.ylabel('Số lượng giao dịch')
plt.title('Số lượng giao dịch theo quý')
plt.ylim(0, ax.get_ylim()[1] * 1.12)
plt.xticks(ticks=range(len(quarterly_counts)), labels=quarterly_counts.index, rotation=45)
plt.show()
```



In [26]:

```
# Sự chênh lệch giữa quý 1 và quý 2
q1_count = quarterly_counts.get('2019Q1', 0)
q2_count = quarterly_counts.get('2019Q2', 0)
q3_count = quarterly_counts.get('2019Q3', 0)
q4_count = quarterly_counts.get('2019Q4', 0)
chenh_lech_q1_q2 = ((q2_count - q1_count) / q1_count) * 100 if q1_count != 0 else 0
chenh_lech_q2_q3 = ((q3_count - q2_count) / q2_count) * 100 if q2_count != 0 else 0
chenh_lech_q3_q4 = ((q4_count - q3_count) / q3_count) * 100 if q3_count != 0 else 0
print(f"Sự chênh lệch giữa quý 1 và quý 2 là: {chenh_lech_q1_q2} %")
print(f"Sự chênh lệch giữa quý 2 và quý 3 là: {chenh_lech_q2_q3} %")
print(f"Sự chênh lệch giữa quý 3 và quý 4 là: {chenh_lech_q3_q4} %")
```

Sự chênh lệch giữa quý 1 và quý 2 là: -85.09406657018813 %

Sự chênh lệch giữa quý 2 và quý 3 là: 12.62135922330097 %

Sự chênh lệch giữa quý 3 và quý 4 là: -22.413793103448278 %

- Phân tích

- Từ quý 1 sang quý 2: cho thấy sự chênh lệch lớn. Nguyên nhân có thể do quý 2 là các tháng nên sức mua của khách hàng giảm
- Từ quý 2 sang quý 3: Có thể thấy từ quý 2 sang quý 3 sức mua tăng lên nhưng không đáng kể 12.6%. Cho thấy siêu thị, các hàng và các sản phẩm đã áp dụng các chính sách để kích cầu n
- Từ quý 3 và quý 4: Sức mua lại giảm so với quý 2 và quý 3. Cho thấy rằng các chính sách chu hiệu quả, các chính sách chỉ hiệu quả lúc ra mắt tức là quý 2 sang quý 3 nhưng sau quý 3 cá sách này lại không thu hút được khách hàng

Đếm số lượng đánh giá theo giao dịch

```
In [27]: # Điểm được đánh giá cao nhất và thấp nhất trong các đánh giá
max_rating = supermarket_data_coppy['Rating'].max()
min_rating = supermarket_data_coppy['Rating'].min()
print(f"Điểm đánh giá cao nhất là: {max_rating}")
print(f"Điểm đánh giá thấp nhất là: {min_rating}")
# ===== CHIA BIN THEO MỨC KINH DOANH =====
bins = [4.0, 5.0, 6.5, 7.5, 8.5, 10.0]
labels = ['Rất kém', 'Kém', 'Trung bình', 'Khá', 'Tốt']

supermarket_data_coppy['Rating Category'] = pd.cut(
    supermarket_data_coppy['Rating'],
    bins=bins,
    labels=labels,
    include_lowest=True
)
```

Điểm đánh giá cao nhất là: 10.0

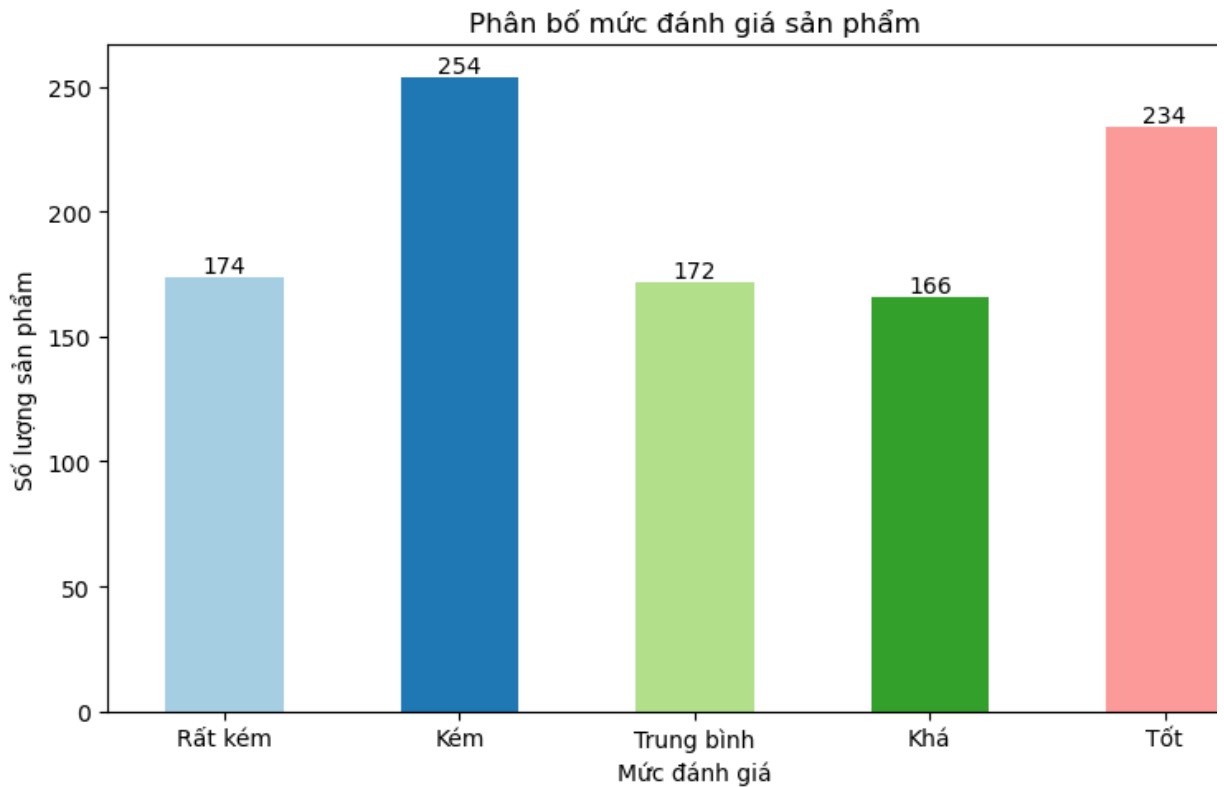
Điểm đánh giá thấp nhất là: 4.0

```
In [28]: # ===== VẼ BIỂU ĐỒ =====
rating_counts = supermarket_data_coppy['Rating Category'].value_counts().sort_index()
ax = rating_counts.plot(
    kind='bar',
    figsize=(8, 5),
    color=plt.cm.Paired.colors
)

plt.xlabel('Mức đánh giá')
plt.ylabel('Số lượng sản phẩm')
plt.title('Phân bố mức đánh giá sản phẩm')
plt.xticks(rotation=0)

# ===== HIỂN THỊ SỐ LƯỢNG TRÊN ĐẦU CỘT =====
for i, value in enumerate(rating_counts):
    ax.text(
        i,
        value,
        str(value),
        ha='center',
        va='bottom',
        fontsize=10
    )

plt.tight_layout()
plt.show()
```



- Phân tích:

- Dựa vào biểu đồ đánh giá cho thấy siêu thị có nhiều sản phẩm bị đánh giá kém nhiều nhất và lượt đánh
- Nên cải thiện chất lượng sản phẩm
- Chưa có nhiều sản phẩm được đánh giá ở trên mức trung bình, các sản phẩm đa số được để mức trung bình trở xuống
- Mỗi một thành phố sẽ có 1 hãng khác nhau bán hàng

Phân tích tổng quan kinh doanh

Doanh thu theo thành phố và chi nhánh

```
In [29]: branch_sales = supermarket_data_coppy.groupby(['Branch', 'City'])['Total'].sum()

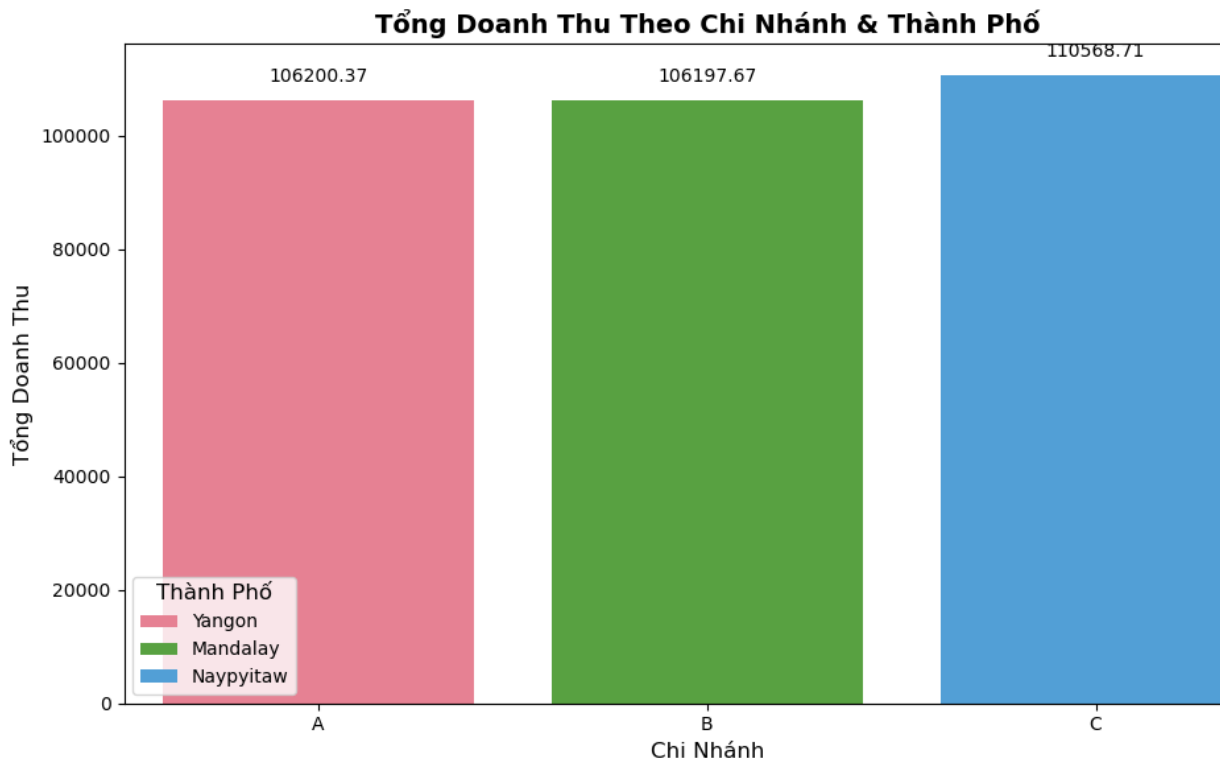
plt.figure(figsize=(10, 6))

ax = sns.barplot(
    data = branch_sales,
    x = 'Branch',
    y = 'Total',
    hue='City',
    palette='husl'
)
```

```
plt.title('Tổng Doanh Thu Theo Chi Nhánh & Thành Phố', fontsize=14, fontweight='bold')
plt.xlabel('Chi Nhánh', fontsize=12)
plt.ylabel('Tổng Doanh Thu', fontsize=12)

for container in ax.containers:
    ax.bar_label(container, fmt='%.2f', padding=8, fontsize=10)

plt.legend(title='Thành Phố', title_fontsize=12, fontsize=10)
plt.tight_layout()
plt.show()
```



```
In [30]: # Tổng doanh thu cho cả ba chi nhánh
total_sales = branch_sales['Total'].sum()
print(f"Tổng doanh thu cho cả ba chi nhánh là: {total_sales:.2f}")
```

Tổng doanh thu cho cả ba chi nhánh là: 322966.75

- Phân tích:

- Chi nhánh C đang có doanh thu cao nhất
- Chi nhánh A và B có doanh thu ngang nhau không có sự chênh lệch quá lớn
- Chi nhánh B đang có doanh thu thấp nhất
- Mỗi một chi nhánh nằm trong 1 thành phố duy nhất
- Tại sao chi nhánh C lại có doanh thu cao nhất trong cả ba chi nhánh trong khi đó chi nhánh A và B có doanh thu tương đương nhau?

- Nhìn chung doanh thu của cả ba chi nhánh khá đồng đều không có sự chênh lệch quá lớn giữa các chi nhánh

Gross income theo chi nhánh

```
In [31]: gross_branch_city = supermarket_data_coppy.groupby(['Branch', 'City'])['gross income']

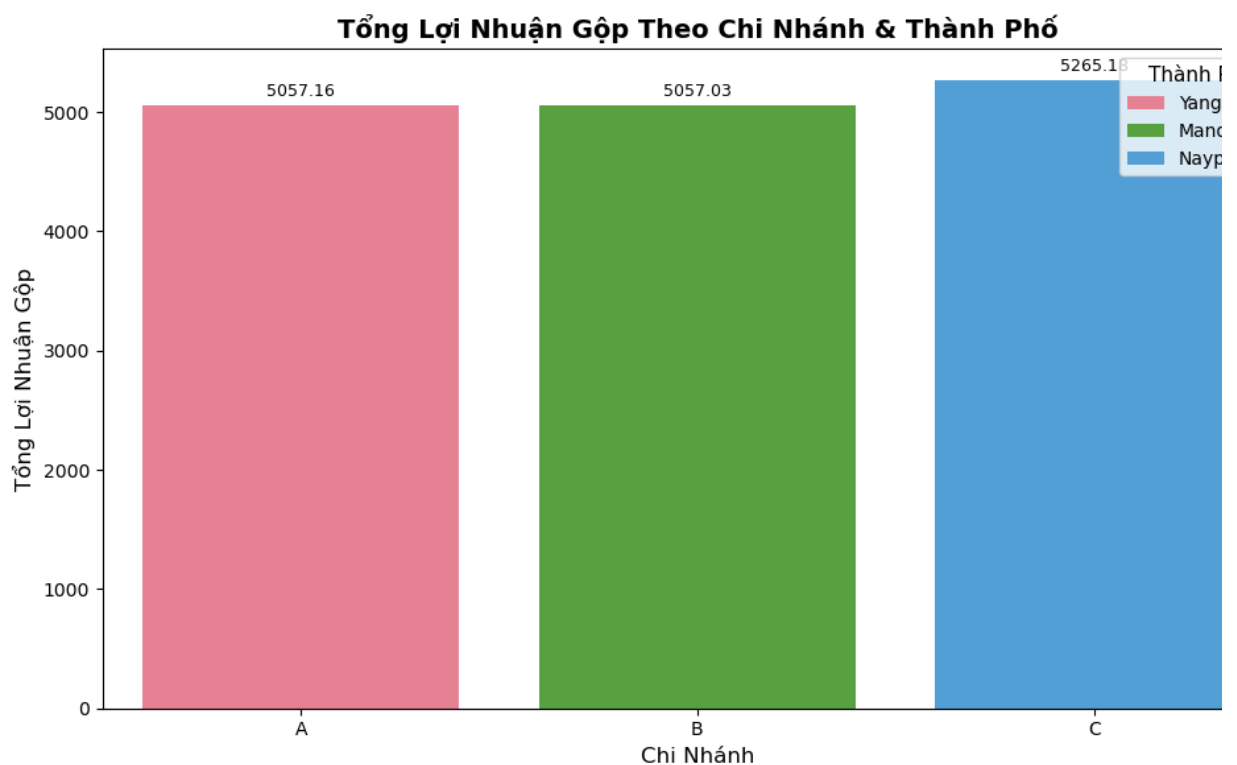
plt.figure(figsize=(10, 6))

ax = sns.barplot(
    data=gross_branch_city,
    x='Branch',
    y='gross income',
    hue='City',
    palette='husl',
    errorbar=None # nếu không cần error bar
)

plt.title('Tổng Lợi Nhuận Gộp Theo Chi Nhánh & Thành Phố', fontsize=14, fontweight='bold')
plt.xlabel('Chi Nhánh', fontsize=12)
plt.ylabel('Tổng Lợi Nhuận Gộp', fontsize=12)

# Hiển thị nhãn số trên mỗi thanh
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f', padding=3, fontsize=9)

plt.legend(title='Thành Phố', title_fontsize=11, fontsize=10, loc='upper right')
plt.tight_layout()
plt.show()
```



- Phân tích:

- Chi nhánh C đang có lợi nhuận gộp cao nhất
- Chi nhánh A và B lợi nhuận gộp ngang nhau do doanh thu của hai chi nhánh này ngang nhau

- Chi nhánh B đang có lợi nhuận thấp nhất
 - Tại sao chi nhánh C lại có lợi nhuận cao nhất trong cả ba chi nhánh trong khi đó chi nhánh A doanh thu tương đương nhau?
- Nhìn chung lợi nhuận gộp của cả ba chi nhánh khá đồng đều không có sự chênh lệch quá lớn ở nhánh

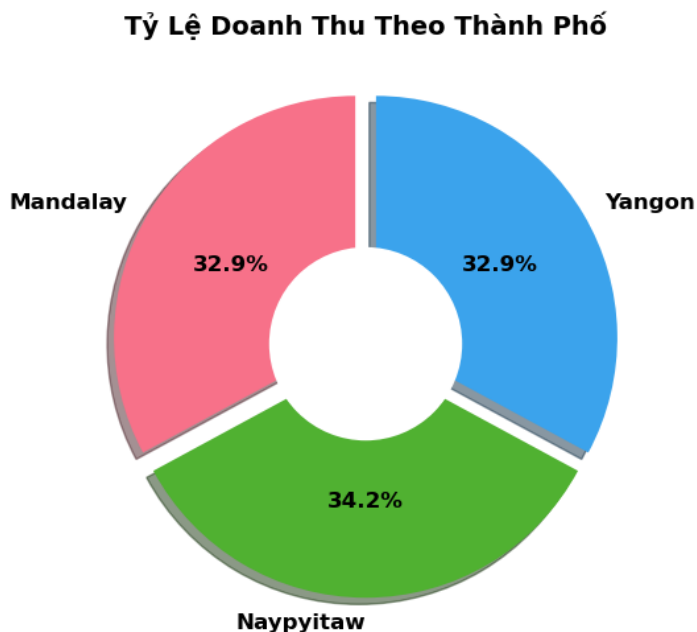
Phân bổ doanh thu theo thành phố

```
In [32]: city_sales = supermarket_data_coppy.groupby('City')['Total'].sum().reset_index()
plt.figure(figsize=(10, 5))

# Vẽ pie
plt.pie(
    city_sales['Total'],
    labels=city_sales['City'],
    autopct='%1.1f%%',          # hiển thị phần trăm + 1 chữ số thập phân
    startangle=90,
    colors=sns.color_palette('husl', len(city_sales)), # màu đẹp
    shadow=True,
    explode=[0.05] * len(city_sales), # tách nhẹ các miếng
    textprops={'fontsize': 12, 'fontweight': 'bold'}
)

# Thêm tiêu đề và vòng tròn giữa (donut style giống px)
centre_circle = plt.Circle((0,0), 0.4, fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)

plt.title('Tỷ Lệ Doanh Thu Theo Thành Phố', fontsize=14, fontweight='bold', pad=)
plt.axis('equal') # giữ hình tròn
plt.tight_layout()
plt.show()
```



- Phân tích:

- Tỷ lệ doanh thu giữa ba chi nhánh không có sự chênh lệch quá lớn
- Thành phố Mandalay và Yangon có tỷ lệ bằng nhau
- Thành phố Naypyitaw đóng góp doanh thu cao nhất trong cả ba thành phố

Phân tích sản phẩm

Số lượng sản phẩm bán ra

```
In [33]: product_line_sales = supermarket_data_coppy.groupby('Product line')['Quantity'].
product_line_sales = product_line_sales.sort_values('Quantity', ascending=False)

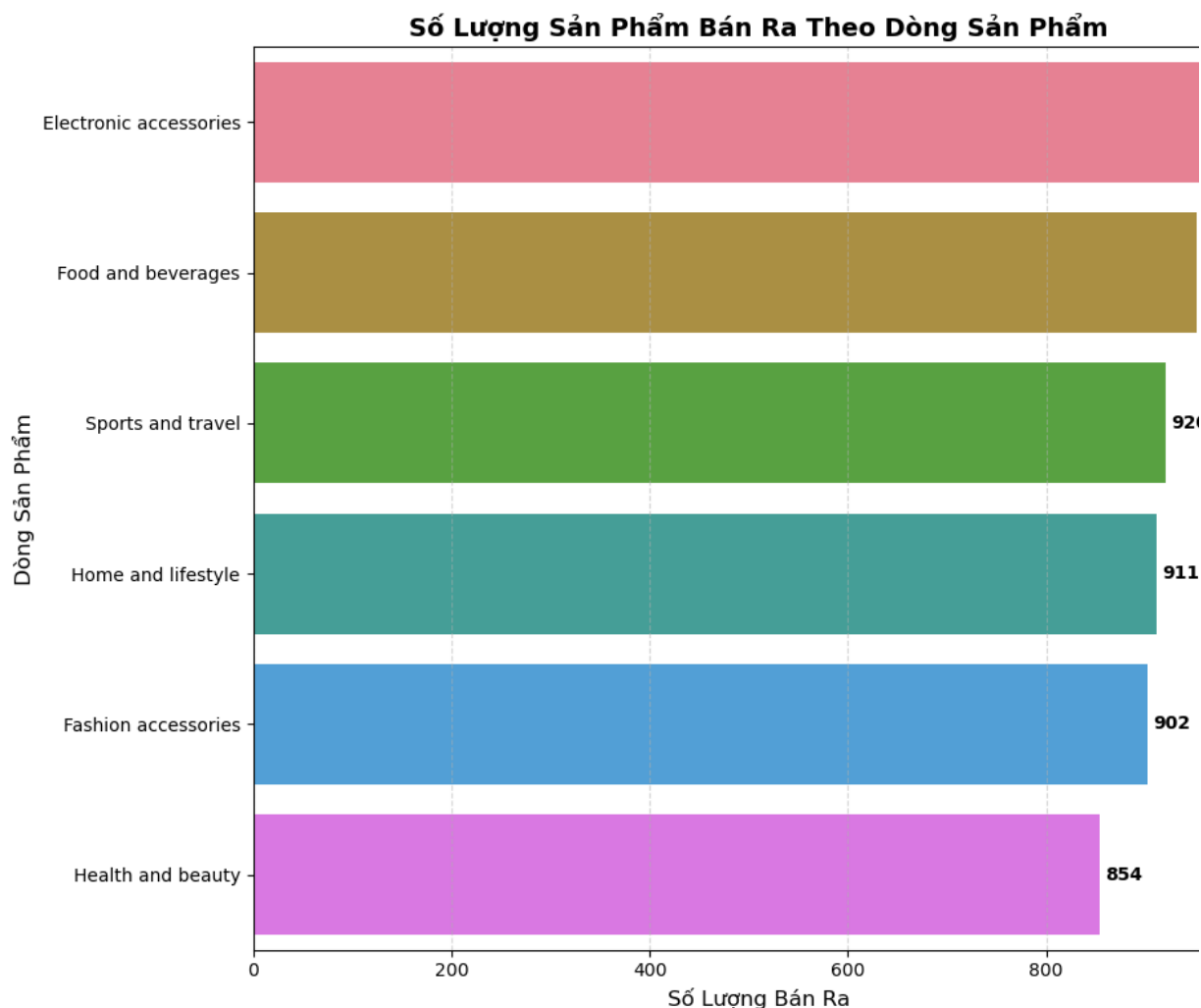
plt.figure(figsize=(10, 8)) # Kích thước phù hợp cho ngang (cao hơn để chứa nhi

ax = sns.barplot(
    data=product_line_sales,
    x='Quantity',           # Hoán đổi: x Là Quantity để thanh ngang
    y='Product line',       # y Là Product Line
    palette='husl',
    hue='Product line',     # Để màu khác nhau theo dòng sản phẩm
    legend=False            # Không cần Legend
)

plt.title('Số Lượng Sản Phẩm Bán Ra Theo Dòng Sản Phẩm', fontsize=14, fontweight
plt.xlabel('Số Lượng Bán Ra', fontsize=12)
plt.ylabel('Dòng Sản Phẩm', fontsize=12)

# Hiển thị số trên mỗi thanh (định dạng số nguyên, đặt vị trí phù hợp cho ngang)
for container in ax.containers:
    ax.bar_label(container, fmt='%d', padding=3, fontsize=10, fontweight='bold')

plt.yticks(fontsize=10) # Font cho Label y (không cần xoay vì ngang)
plt.grid(axis='x', linestyle='--', alpha=0.5) # Grid theo x vì ngang
plt.tight_layout()
plt.show()
```

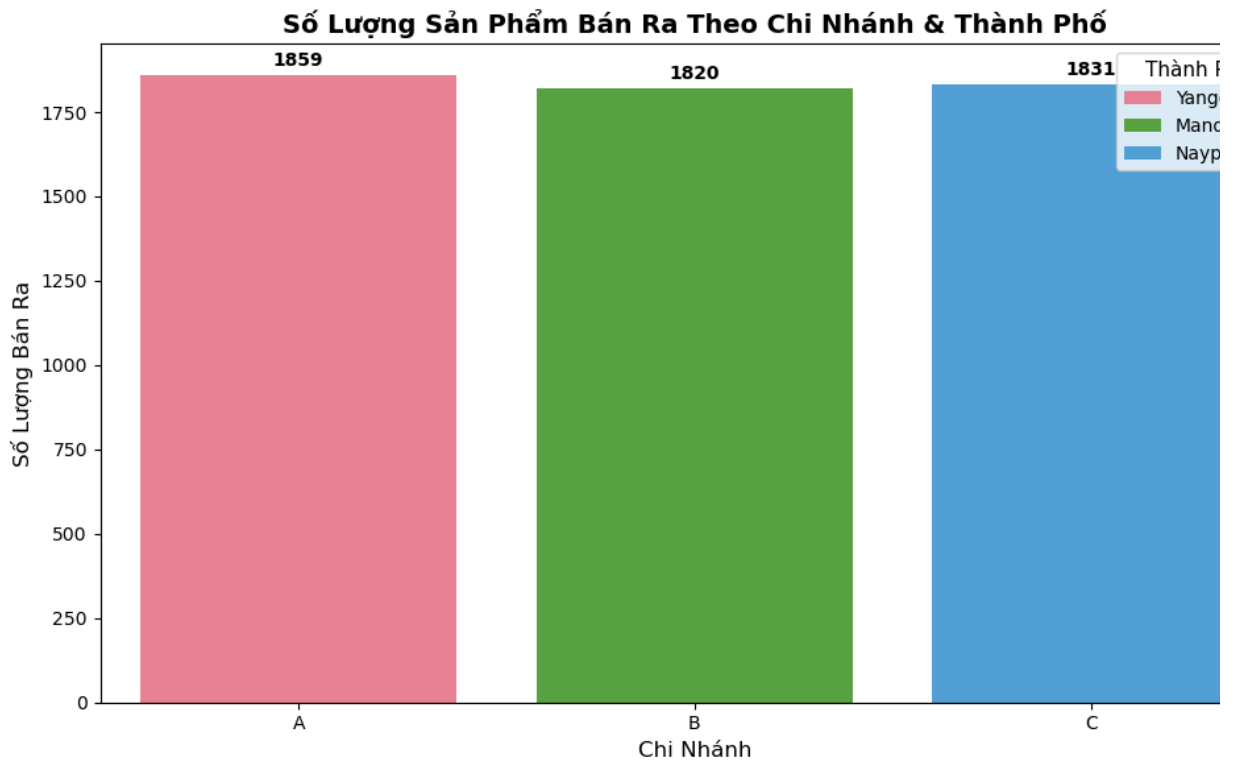


```
In [34]: # tổng số Lượng bán ra của tất cả các dòng sản phẩm
total_quantity = product_line_sales['Quantity'].sum()
print(f"Tổng số lượng bán ra của tất cả các dòng sản phẩm là: {total_quantity}")
```

Tổng số lượng bán ra của tất cả các dòng sản phẩm là: 5510

Số lượng sản phẩm bán ra theo chi nhánh/thành phố

```
In [35]: # số Lượng sản bán ra theo chi nhánh/thành phố
branch_city_quantity = supermarket_data_coppy.groupby(['Branch', 'City'])['Quantity']
plt.figure(figsize=(10, 6))
ax = sns.barplot(
    data=branch_city_quantity,
    x='Branch',
    y='Quantity',
    hue='City',
    palette='husl',
    errorbar=None
)
plt.title('Số Lượng Sản Phẩm Bán Ra Theo Chi Nhánh & Thành Phố', fontsize=14, fontweight='bold')
plt.xlabel('Chi Nhánh', fontsize=12)
plt.ylabel('Số Lượng Bán Ra', fontsize=12)
for container in ax.containers:
    ax.bar_label(container, fmt='%d', padding=3, fontsize=10, fontweight='bold')
plt.legend(title='Thành Phố', title_fontsize=11, fontsize=10, loc='upper right')
plt.tight_layout()
plt.show()
```



Số lượng dòng sản phẩm bán ra theo chi nhánh A

```
In [36]: # Lọc và groupby như trên
branch_a_data = supermarket_data_coppy[supermarket_data_coppy['Branch'] == 'A']
product_line_a = branch_a_data.groupby('Product line')['Quantity'].sum().reset_index()
product_line_a = product_line_a.sort_values('Quantity', ascending=False)

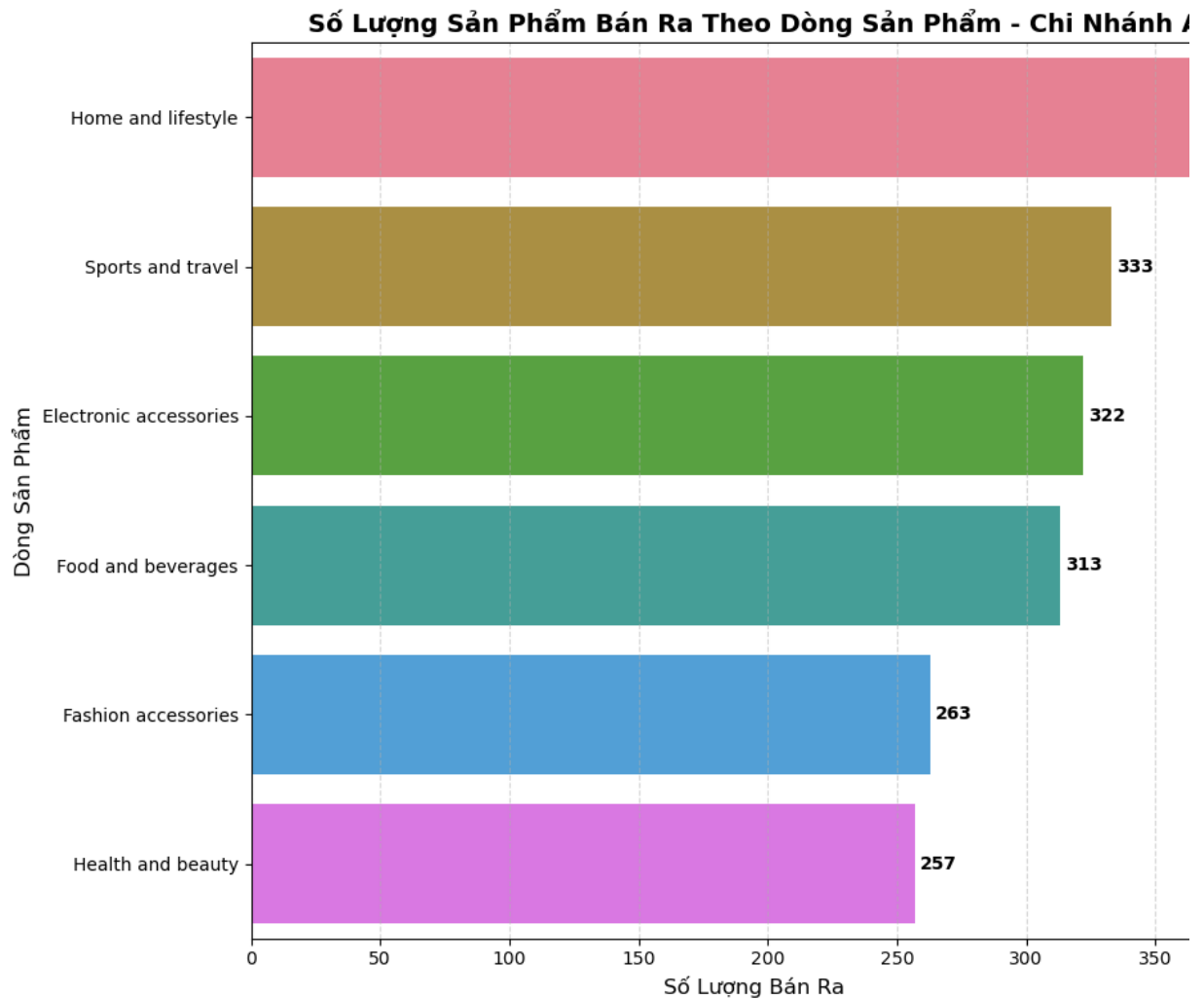
# Vẽ ngang
plt.figure(figsize=(10, 8)) # Cao hơn để chứa tên sản phẩm

ax = sns.barplot(
    data=product_line_a,
    x='Quantity',
    y='Product line',
    palette='husl',
    hue='Product line',
    legend=False
)

plt.title('Số Lượng Sản Phẩm Bán Ra Theo Dòng Sản Phẩm - Chi Nhánh A',
          fontsize=14, fontweight='bold')
plt.xlabel('Số Lượng Bán Ra', fontsize=12)
plt.ylabel('Dòng Sản Phẩm', fontsize=12)

# Thêm nhãn số bên phải thanh
for container in ax.containers:
    ax.bar_label(container, fmt='%d', padding=3, fontsize=10, fontweight='bold')

plt.yticks(fontsize=10)
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Số lượng dòng sản phẩm bán ra theo chi nhánh B

```
In [37]: # Lọc và groupby như trên
branch_b_data = supermarket_data_copy[supermarket_data_copy['Branch'] == 'B']
product_line_b = branch_b_data.groupby('Product line')['Quantity'].sum().reset_index()
product_line_b = product_line_b.sort_values('Quantity', ascending=False)

# Vẽ ngang
plt.figure(figsize=(10, 8)) # Cao hơn để chứa tên sản phẩm

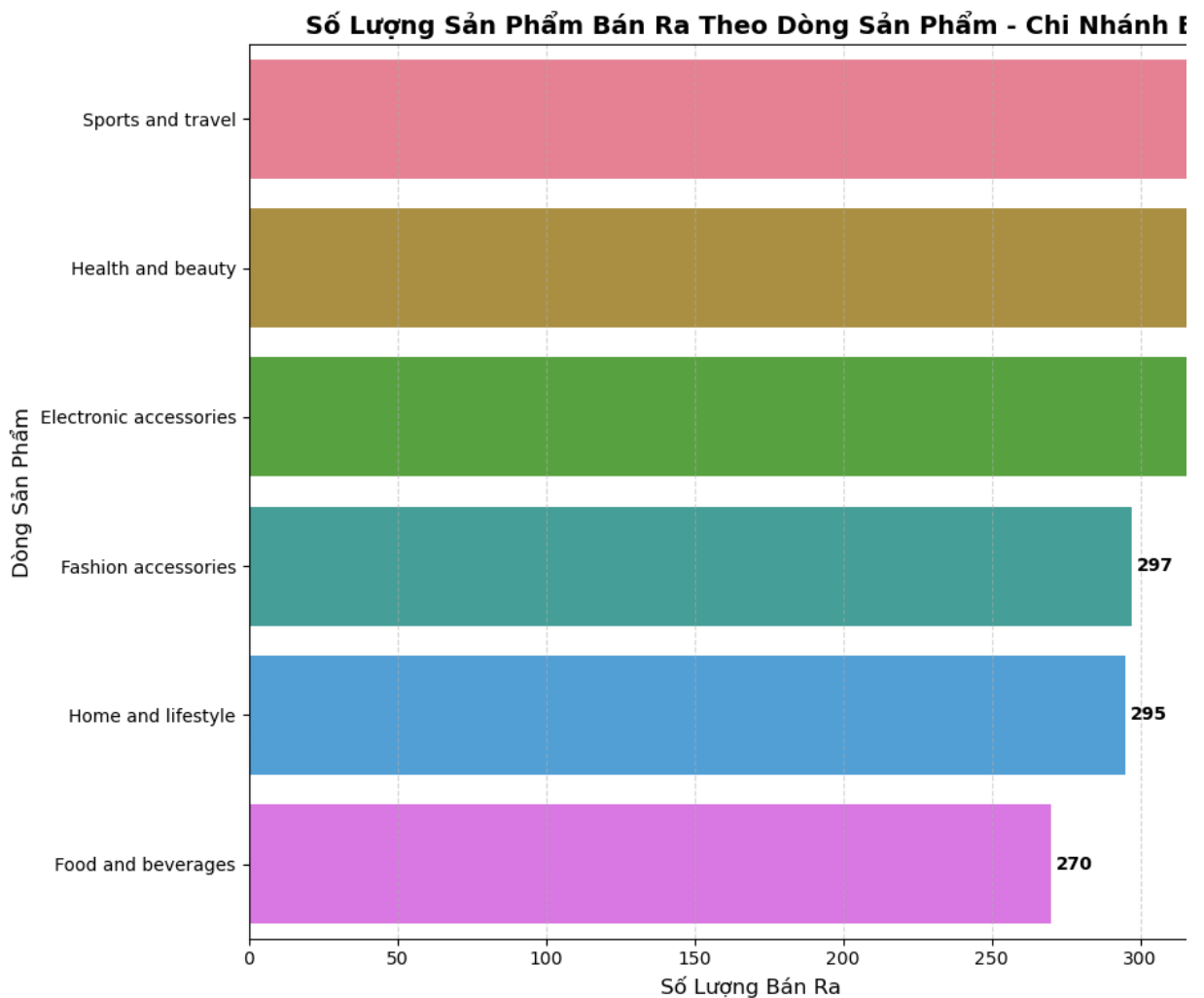
ax = sns.barplot(
    data=product_line_b,
    x='Quantity',
    y='Product line',
    palette='husl',
    hue='Product line',
    legend=False
)

plt.title('Số Lượng Sản Phẩm Bán Ra Theo Dòng Sản Phẩm - Chi Nhánh B',
          fontsize=14, fontweight='bold')
plt.xlabel('Số Lượng Bán Ra', fontsize=12)
plt.ylabel('Dòng Sản Phẩm', fontsize=12)

# Thêm nhãn số bên phải thanh
for container in ax.containers:
```

```
ax.bar_label(container, fmt='%d', padding=3, fontsize=10, fontweight='bold')

plt.yticks(fontsize=10)
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Số lượng dòng sản phẩm bán ra theo chi nhánh C

```
In [38]: # Lọc và groupby như trên
branch_c_data = supermarket_data_coppy[supermarket_data_coppy['Branch'] == 'C']
product_line_c = branch_c_data.groupby('Product line')['Quantity'].sum().reset_index()
product_line_c = product_line_c.sort_values('Quantity', ascending=False)

# Vẽ ngang
plt.figure(figsize=(10, 8)) # Cao hơn để chứa tên sản phẩm

ax = sns.barplot(
    data=product_line_c,
    x='Quantity',
    y='Product line',
    palette='husl',
    hue='Product line',
    legend=False
)

plt.title('Số Lượng Sản Phẩm Bán Ra Theo Dòng Sản Phẩm - Chi Nhánh C',
```

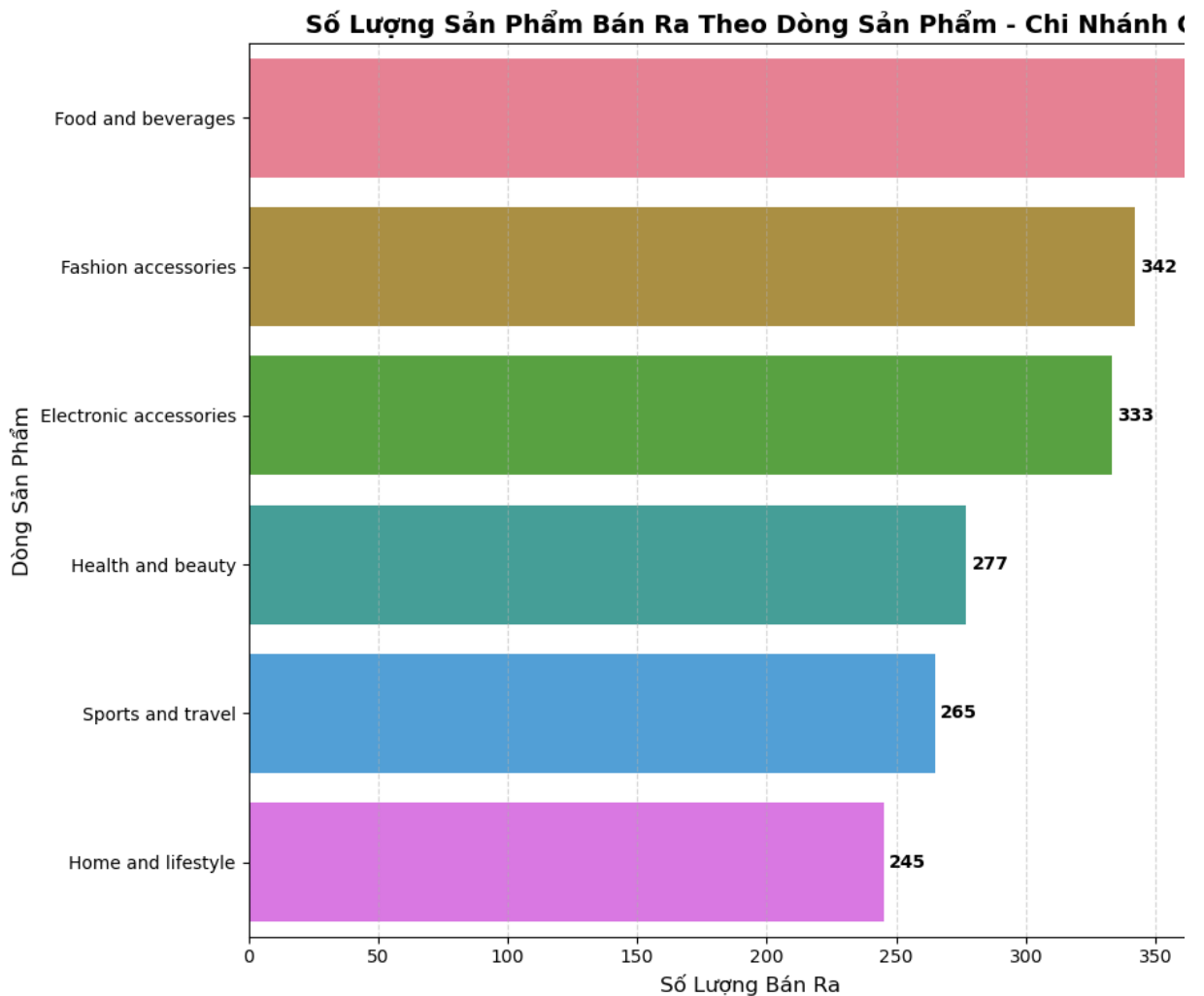
```

        fontsize=14, fontweight='bold')
plt.xlabel('Số Lượng Bán Ra', fontsize=12)
plt.ylabel('Dòng Sản Phẩm', fontsize=12)

# Thêm nhãn số bên phải thanh
for container in ax.containers:
    ax.bar_label(container, fmt='%d', padding=3, fontsize=10, fontweight='bold')

plt.yticks(fontsize=10)
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```



Doanh thu theo từng sản phẩm

```

In [39]: # 1. Tính tổng doanh thu (Total) theo từng dòng sản phẩm
product_sales = supermarket_data_coppy.groupby('Product line')['Total'].sum().re

# 2. Sắp xếp từ cao đến thấp để cột ngang có thứ tự rõ ràng
product_sales = product_sales.sort_values('Total', ascending=False)

# 3. Xác định sản phẩm cao nhất và thấp nhất
highest_product = product_sales.iloc[0]
lowest_product = product_sales.iloc[-1]

print(f"Sản phẩm có doanh thu CAO NHẤT: {highest_product['Product line']}")
print(f"Doanh thu: {highest_product['Total']:, .2f}")

```

```

print(f"Sản phẩm có doanh thu THẤP NHẤT: {lowest_product['Product line']}")
print(f"Doanh thu: {lowest_product['Total']:, .2f}")

# 4. Vẽ biểu đồ cột NGANG
plt.figure(figsize=(10, 8)) # Kích thước phù hợp cho tên sản phẩm dài

ax = sns.barplot(
    data=product_sales,
    x='Total',
    y='Product line',
    palette='viridis', # palette đẹp, thay 'husl' nếu muốn
    hue='Product line',
    legend=False
)

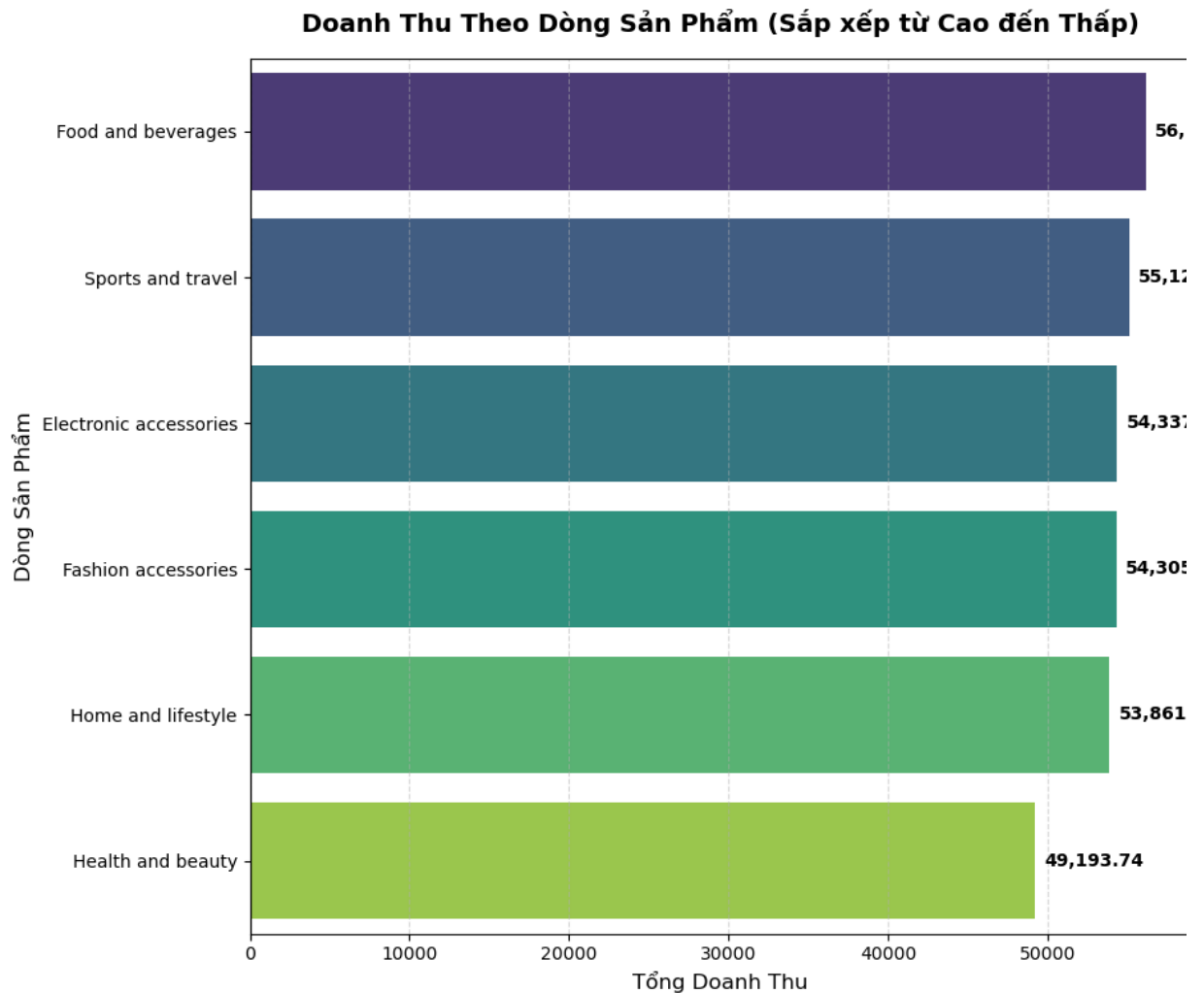
# Thêm nhãn số trên thanh (định dạng có dấu phẩy, 2 chữ số thập phân)
for container in ax.containers:
    ax.bar_label(container, fmt='{:, .2f}', padding=5, fontsize=10, fontweight='b')

plt.title('Doanh Thu Theo Dòng Sản Phẩm (Sắp xếp từ Cao đến Thấp)',
          fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Tổng Doanh Thu', fontsize=12)
plt.ylabel('Dòng Sản Phẩm', fontsize=12)

plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```

Sản phẩm có doanh thu CAO NHẤT: Food and beverages
Doanh thu: 56,144.84
Sản phẩm có doanh thu THẤP NHẤT: Health and beauty
Doanh thu: 49,193.74



Doanh thu theo từng dòng sản phẩm của chi nhánh A

```
In [40]: # Lọc và groupby như trên
branch_a_data = supermarket_data_coppy[supermarket_data_coppy['Branch'] == 'A']
product_line_a = branch_a_data.groupby('Product line')['Total'].sum().reset_index()
product_line_a = product_line_a.sort_values('Total', ascending=False)

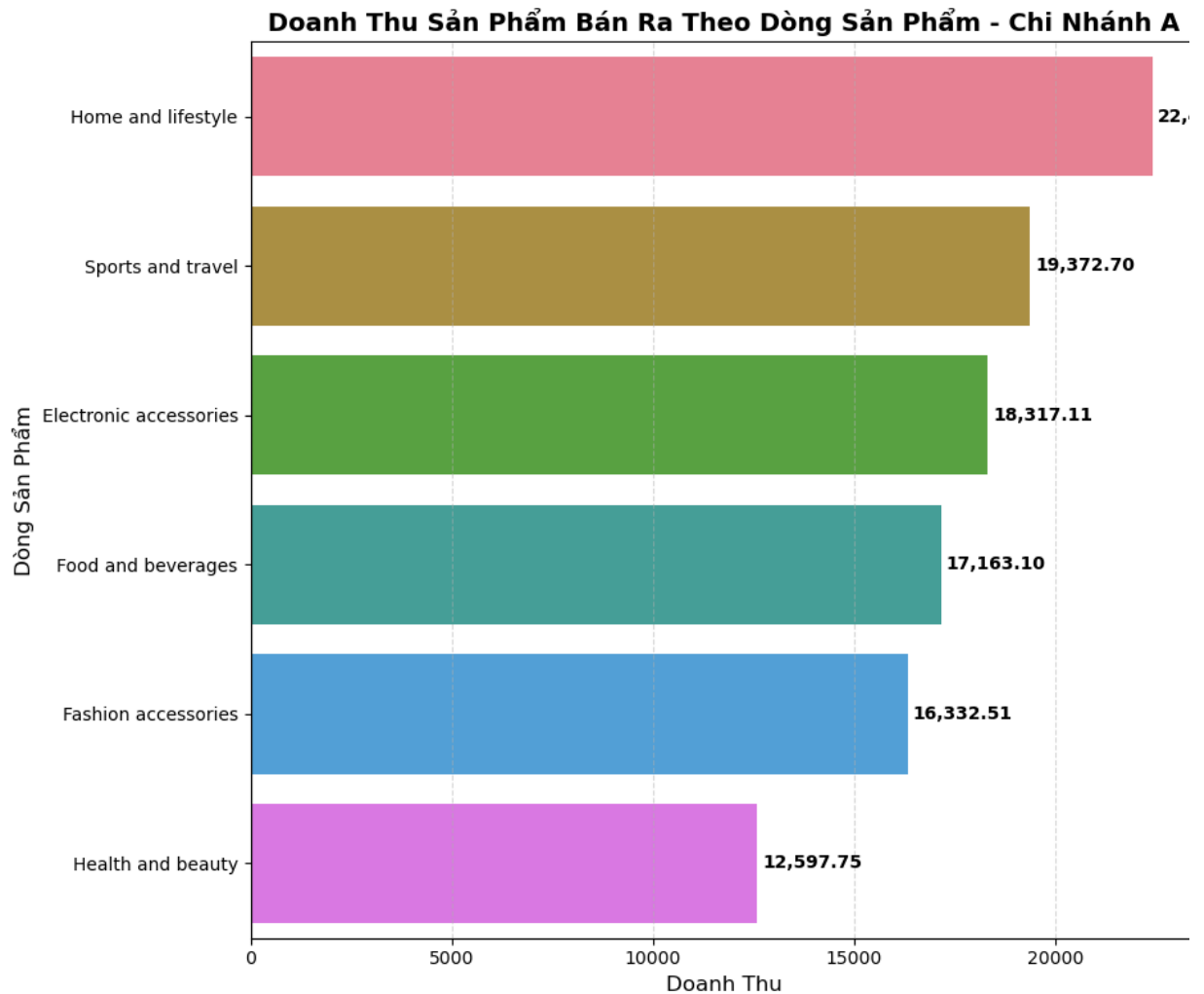
# Vẽ ngang
plt.figure(figsize=(10, 8)) # Cao hơn để chứa tên sản phẩm

ax = sns.barplot(
    data=product_line_a,
    x='Total',
    y='Product line',
    palette='husl',
    hue='Product line',
    legend=False
)

plt.title('Doanh Thu Sản Phẩm Bán Ra Theo Dòng Sản Phẩm - Chi Nhánh A',
          fontsize=14, fontweight='bold')
plt.xlabel('Doanh Thu', fontsize=12)
plt.ylabel('Dòng Sản Phẩm', fontsize=12)

# Thêm nhãn số bên phải thanh
for container in ax.containers:
    ax.bar_label(container, fmt='{:,.2f}', padding=3, fontsize=10, fontweight='b')
```

```
plt.yticks(fontsize=10)
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Doanh thu theo từng dòng sản phẩm của chi nhánh B

```
In [41]: # Lọc và groupby như trên
branch_b_data = supermarket_data_coppy[supermarket_data_coppy['Branch'] == 'B']
product_line_b = branch_b_data.groupby('Product line')['Total'].sum().reset_index()
product_line_b = product_line_b.sort_values('Total', ascending=False)

# Vẽ ngang
plt.figure(figsize=(10, 8)) # Cao hơn để chứa tên sản phẩm

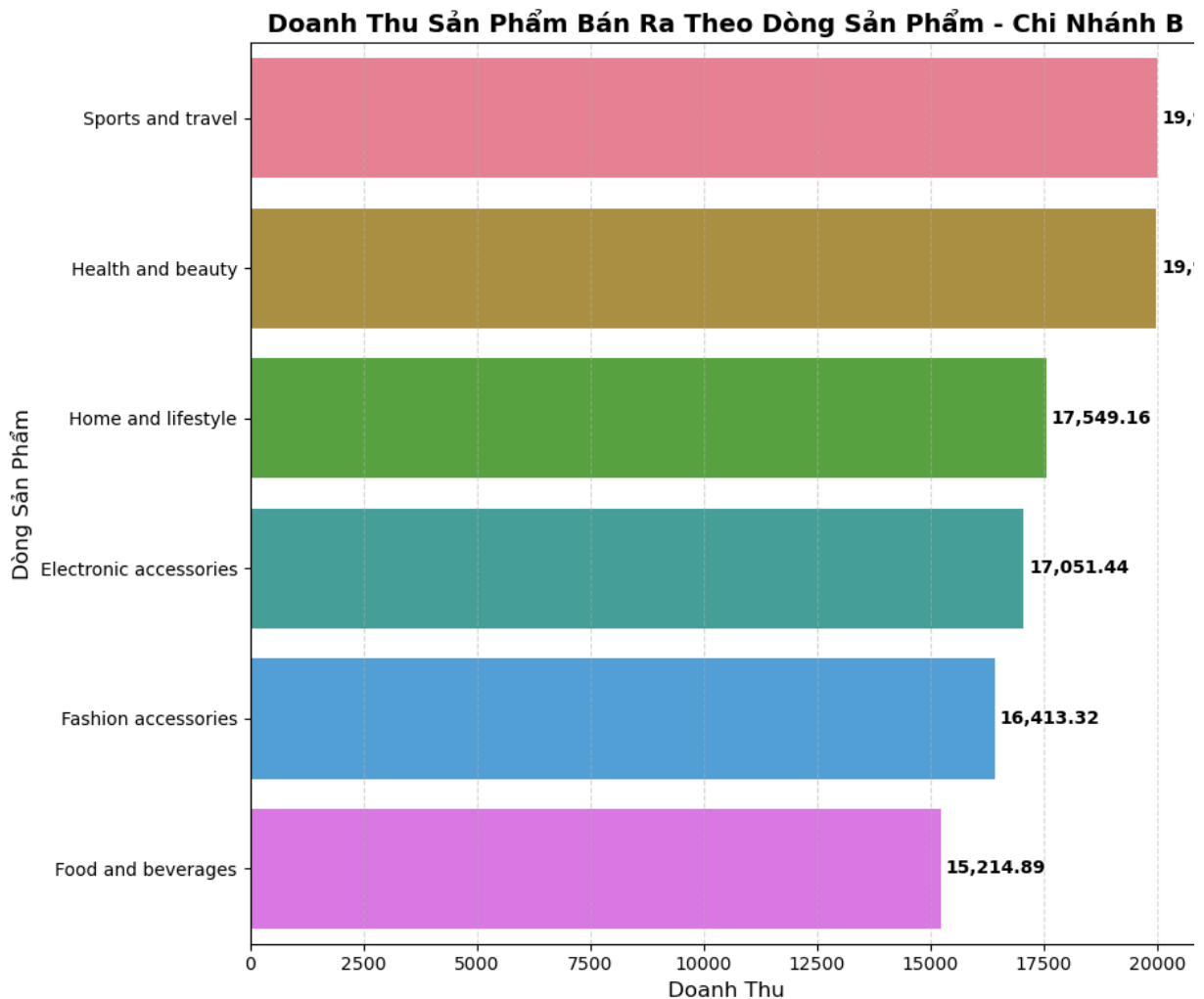
ax = sns.barplot(
    data=product_line_b,
    x='Total',
    y='Product line',
    palette='husl',
    hue='Product line',
    legend=False
)

plt.title('Doanh Thu Sản Phẩm Bán Ra Theo Dòng Sản Phẩm - Chi Nhánh B',
          fontsize=14, fontweight='bold')
```

```
plt.xlabel('Doanh Thu', fontsize=12)
plt.ylabel('Dòng Sản Phẩm', fontsize=12)

# Thêm nhãn số bên phải thanh
for container in ax.containers:
    ax.bar_label(container, fmt='{:, .2f}', padding=3, fontsize=10, fontweight='b')

plt.yticks(fontsize=10)
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Doanh thu theo từng dòng sản phẩm của chi nhánh C

```
In [42]: # Lọc và groupby như trên
branch_c_data = supermarket_data_coppy[supermarket_data_coppy['Branch'] == 'C']
product_line_c = branch_c_data.groupby('Product line')['Total'].sum().reset_index()
product_line_c = product_line_c.sort_values('Total', ascending=False)

# Vẽ ngang
plt.figure(figsize=(10, 8)) # Cao hơn để chứa tên sản phẩm

ax = sns.barplot(
    data=product_line_c,
    x='Total',
    y='Product line',
    palette='husl',
```

```

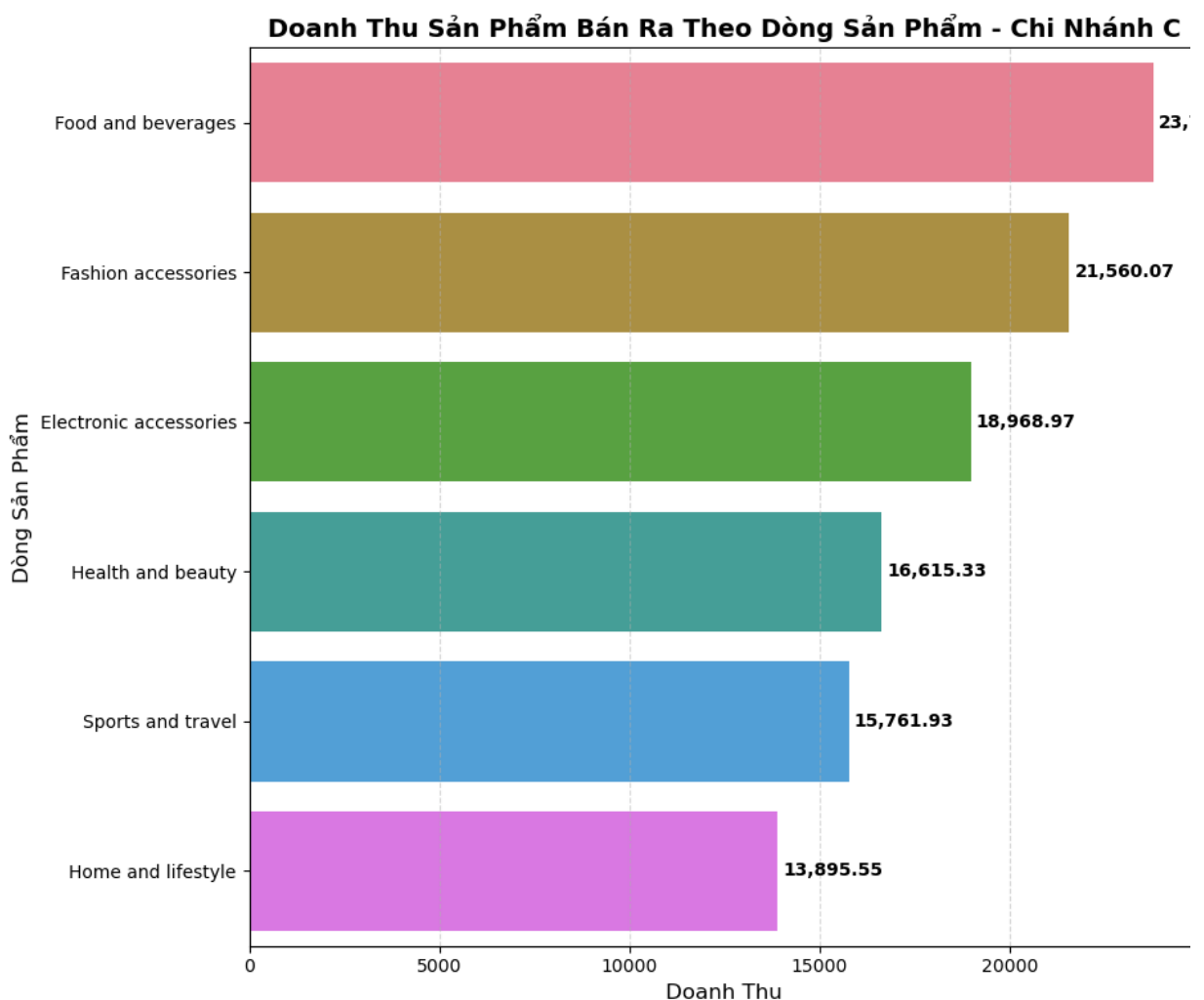
    hue='Product line',
    legend=False
)

plt.title('Doanh Thu Sản Phẩm Bán Ra Theo Dòng Sản Phẩm - Chi Nhánh C',
          fontsize=14, fontweight='bold')
plt.xlabel('Doanh Thu', fontsize=12)
plt.ylabel('Dòng Sản Phẩm', fontsize=12)

# Thêm nhãn số bên phải thanh
for container in ax.containers:
    ax.bar_label(container, fmt='{:, .2f}', padding=3, fontsize=10, fontweight='b')

plt.yticks(fontsize=10)
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```



Phân Tích Theo Thời Gian

Xu hướng doanh thu theo ngày

```

In [43]: supermarket_data_copy['Date'] = pd.to_datetime(supermarket_data_copy['Date'])
supermarket_data_copy['Month'] = supermarket_data_copy['Date'].dt.to_period('M')

```

```

daily_sales = supermarket_data_coppy.groupby(['Date', 'Month'])['Total'].sum().r

# Sắp xếp theo ngày để đường line mượt
daily_sales = daily_sales.sort_values('Date')

# Vẽ facetgrid
g = sns.FacetGrid(
    data=daily_sales,
    col='Month',
    col_wrap=3,
    height=4,
    aspect=1.5,
    sharey=True,
    sharex=False
)

# Mỗi tháng một subplot
# Nếu nhiều tháng thì wrap xuống dòng

# Chia sẻ trục y để so sánh dễ
# Không chia sẻ trục x vì mỗi tháng ngày khác

g.map(sns.lineplot, 'Date', 'Total', marker='o', color='royalblue')

# Thêm tiêu đề cho từng facet
g.set_titles(col_template="{col_name}")

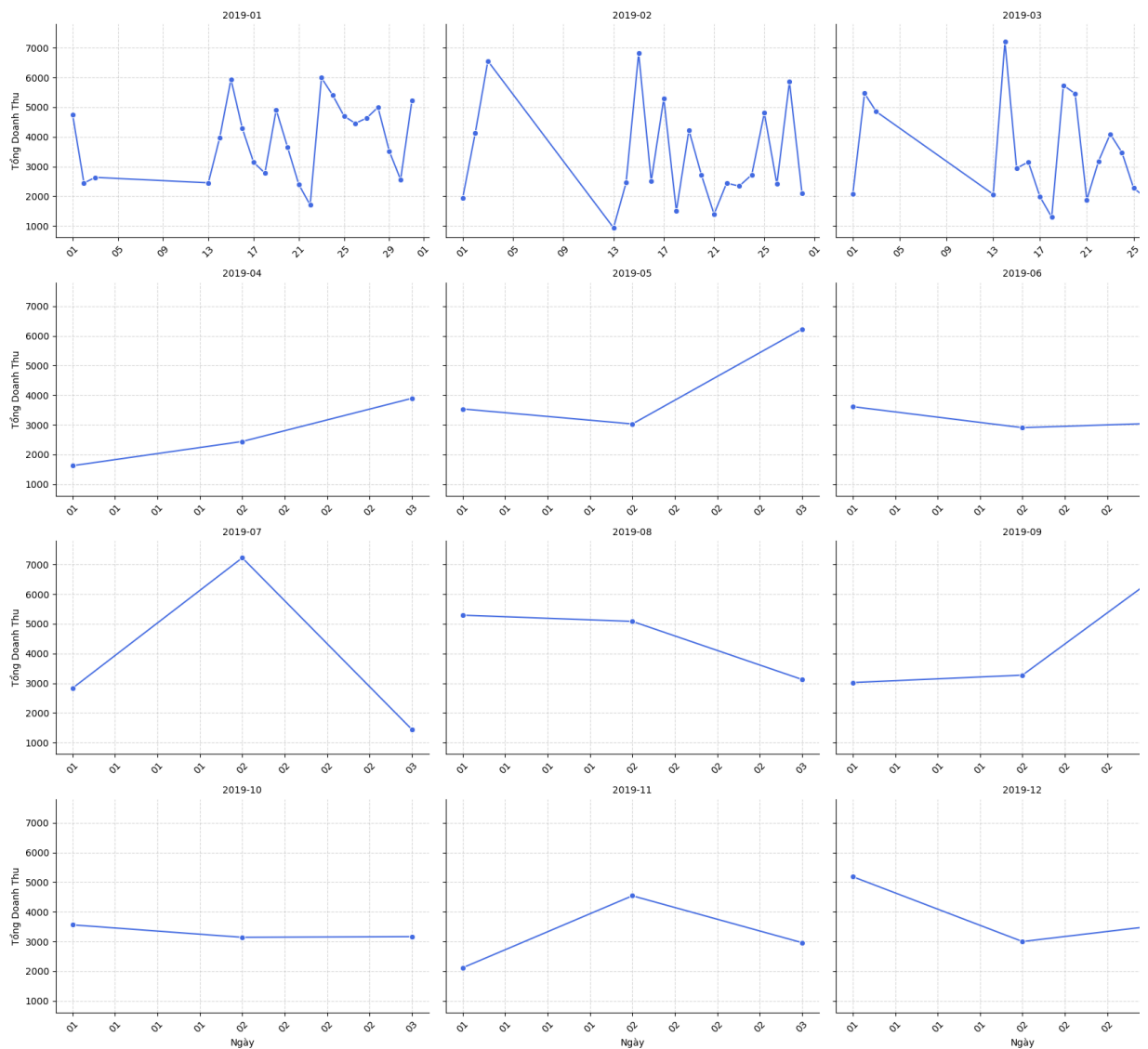
# Format chung
g.set_axis_labels('Ngày', 'Tổng Doanh Thu')
g.set(xticklabels=[]) # Ẩn label ngày nếu muốn gọn, hoặc format riêng bên dưới

# Format ngày cho từng subplot (tùy chọn)
for ax in g.axes.flat:
    ax.xaxis.set_major_formatter(plt.matplotlib.dates.DateFormatter('%d'))
    ax.tick_params(axis='x', rotation=45)
    ax.grid(linestyle='--', alpha=0.5)

g.fig.suptitle('Xu Hướng Doanh Thu Theo Ngày Trong Từng Tháng', fontsize=16, fontweight='bold')
g.tight_layout()
plt.show()

```

Xu Hướng Doanh Thu Theo Ngày Trong Từng Tháng



```
In [44]: g = sns.FacetGrid(
    data=daily_sales,
    col='Month',
    col_wrap=3,
    height=5,
    aspect=1.4,
    sharey=True,
    sharex=False
)

# Vẽ BAR CHART theo ngày
g.map(sns.barplot, 'Date', 'Total', color='cornflowerblue', edgecolor='navy', li

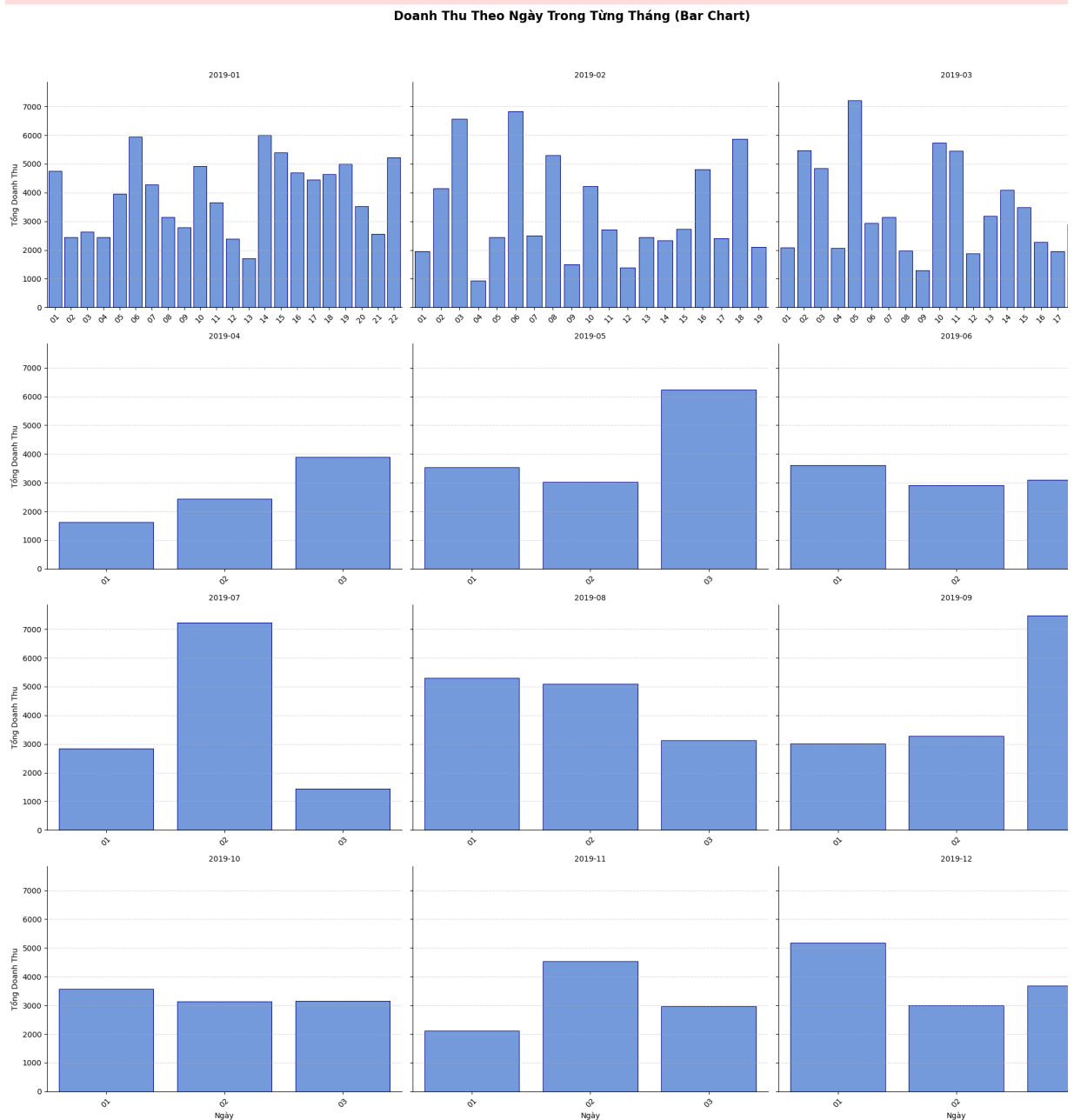
# Format ngày
for ax in g.axes.flat:
    ax.xaxis.set_major_formatter(plt.matplotlib.dates.DateFormatter('%d'))
    ax.tick_params(axis='x', rotation=45)
    ax.grid(axis='y', linestyle='--', alpha=0.5)

g.set_titles(col_template="{col_name}")
g.set_axis_labels('Ngày', 'Tổng Doanh Thu')

g.fig.suptitle('Doanh Thu Theo Ngày Trong Từng Tháng (Bar Chart)',
    fontsize=16, fontweight='bold', y=1.03)
```

```
g.tight_layout()
plt.show()
```

c:\ProgramData\anaconda3\Lib\site-packages\seaborn\axisgrid.py:718: UserWarning: Using barplot function without specifying `order` is likely to produce an incorrect plot.
warnings.warn(warning)



Các ngày có doanh thu cao nhất theo tháng

```
In [45]: # Bước 1: Đảm bảo cột Date là datetime (chạy nếu chưa)
supermarket_data_copy['Date'] = pd.to_datetime(supermarket_data_copy['Date'])

# Bước 2: Tạo cột Month (ví dụ: '2019-01') và Day (ngày trong tháng)
supermarket_data_copy['Month'] = supermarket_data_copy['Date'].dt.to_period('M')
supermarket_data_copy['Day'] = supermarket_data_copy['Date'].dt.day

# Bước 3: Tính tổng doanh thu theo từng ngày trong từng tháng
daily_total = supermarket_data_copy.groupby(['Month', 'Day'])['Total'].sum().re
```

```
# Bước 4: Tìm ngày có doanh thu cao nhất trong từng tháng
# Sắp xếp giảm dần theo Total trong từng tháng, Lấy dòng đầu tiên của mỗi nhóm
daily_total_sorted = daily_total.sort_values(['Month', 'Total'], ascending=[True, False])
highest_day_per_month = daily_total_sorted.groupby('Month').head(1).reset_index()

# Hiển thị kết quả
print("Ngày có doanh thu CAO NHẤT trong từng tháng:")
print(highest_day_per_month[['Month', 'Day', 'Total']])

# Nếu muốn định dạng đẹp hơn
for _, row in highest_day_per_month.iterrows():
    print(f"- Tháng {row['Month']}: Ngày {int(row['Day'])}, Doanh thu = {row['Total']:.2f}")
```

Ngày có doanh thu CAO NHẤT trong từng tháng:

	Month	Day	Total
0	2019-01	23	5994.1875
1	2019-02	15	6830.7855
2	2019-03	14	7214.6340
3	2019-04	3	3894.4395
4	2019-05	3	6230.8785
5	2019-06	1	3614.2050
6	2019-07	2	7228.2105
7	2019-08	1	5293.7325
8	2019-09	3	7474.0470
9	2019-10	1	3560.9490
10	2019-11	2	4542.1530
11	2019-12	1	5184.7635

```
- Tháng 2019-01: Ngày 23, Doanh thu = 5,994.19
- Tháng 2019-02: Ngày 15, Doanh thu = 6,830.79
- Tháng 2019-03: Ngày 14, Doanh thu = 7,214.63
- Tháng 2019-04: Ngày 3, Doanh thu = 3,894.44
- Tháng 2019-05: Ngày 3, Doanh thu = 6,230.88
- Tháng 2019-06: Ngày 1, Doanh thu = 3,614.20
- Tháng 2019-07: Ngày 2, Doanh thu = 7,228.21
- Tháng 2019-08: Ngày 1, Doanh thu = 5,293.73
- Tháng 2019-09: Ngày 3, Doanh thu = 7,474.05
- Tháng 2019-10: Ngày 1, Doanh thu = 3,560.95
- Tháng 2019-11: Ngày 2, Doanh thu = 4,542.15
- Tháng 2019-12: Ngày 1, Doanh thu = 5,184.76
```

Xu hướng doanh thu theo tháng

```
In [46]: monthly_total = supermarket_data_coppy.groupby('Month')['Total'].sum().reset_index()
monthly_total = monthly_total.sort_values('Month')

plt.figure(figsize=(8, 5))
ax = sns.barplot(data=monthly_total, x='Month', y='Total', palette='viridis')

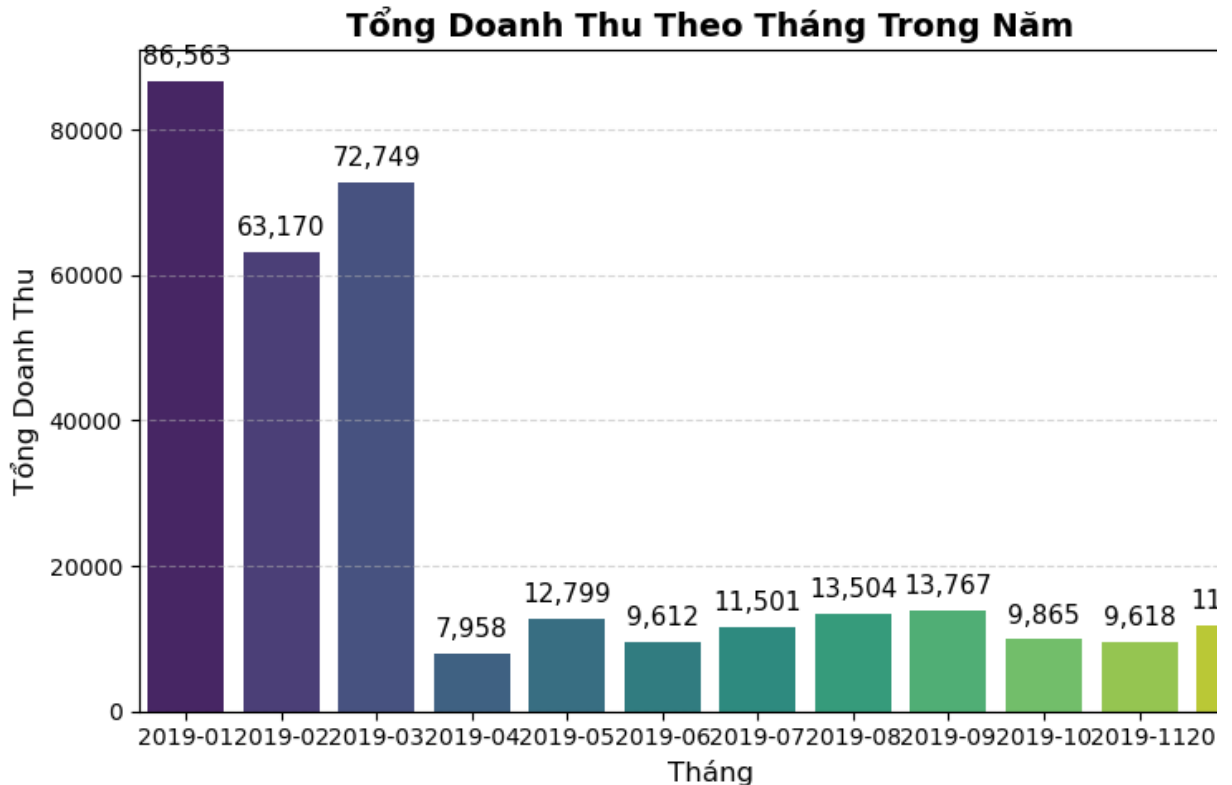
for container in ax.containers:
    ax.bar_label(container, fmt='{:, .0f}', padding=5, fontsize=11)

plt.title('Tổng Doanh Thu Theo Tháng Trong Năm', fontsize=14, fontweight='bold')
plt.xlabel('Tháng', fontsize=12)
plt.ylabel('Tổng Doanh Thu', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_25216\2582131375.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(data=monthly_total, x='Month', y='Total', palette='viridis')
```



Mối quan hệ giữa doanh thu theo giờ trong ngày và ngày trong tuần

```
In [47]: # Tính tổng doanh thu theo giờ (đã có sẵn)
hour_sales = supermarket_data_coppy.groupby('Hour')['Total'].sum().reset_index()

# (Tùy chọn) Loại bỏ giờ không có dữ liệu nếu muốn rõ ràng hơn
hour_sales = hour_sales[hour_sales['Total'] > 0]

# Sắp xếp lại theo giờ (để chắc chắn)
hour_sales = hour_sales.sort_values('Hour')

# Vẽ bar chart
plt.figure(figsize=(10, 6))
bars = plt.bar(hour_sales['Hour'], hour_sales['Total'],
               color='coral', edgecolor='black', width=0.7)

plt.title('Tổng Doanh Thu Theo Giờ Trong Ngày', fontsize=14, pad=15)
plt.xlabel('Giờ trong ngày', fontsize=12)
plt.ylabel('Tổng Doanh Thu', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.5)

# Thêm giá trị lên đầu mỗi cột
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 5000, # điều chỉnh +5000
```

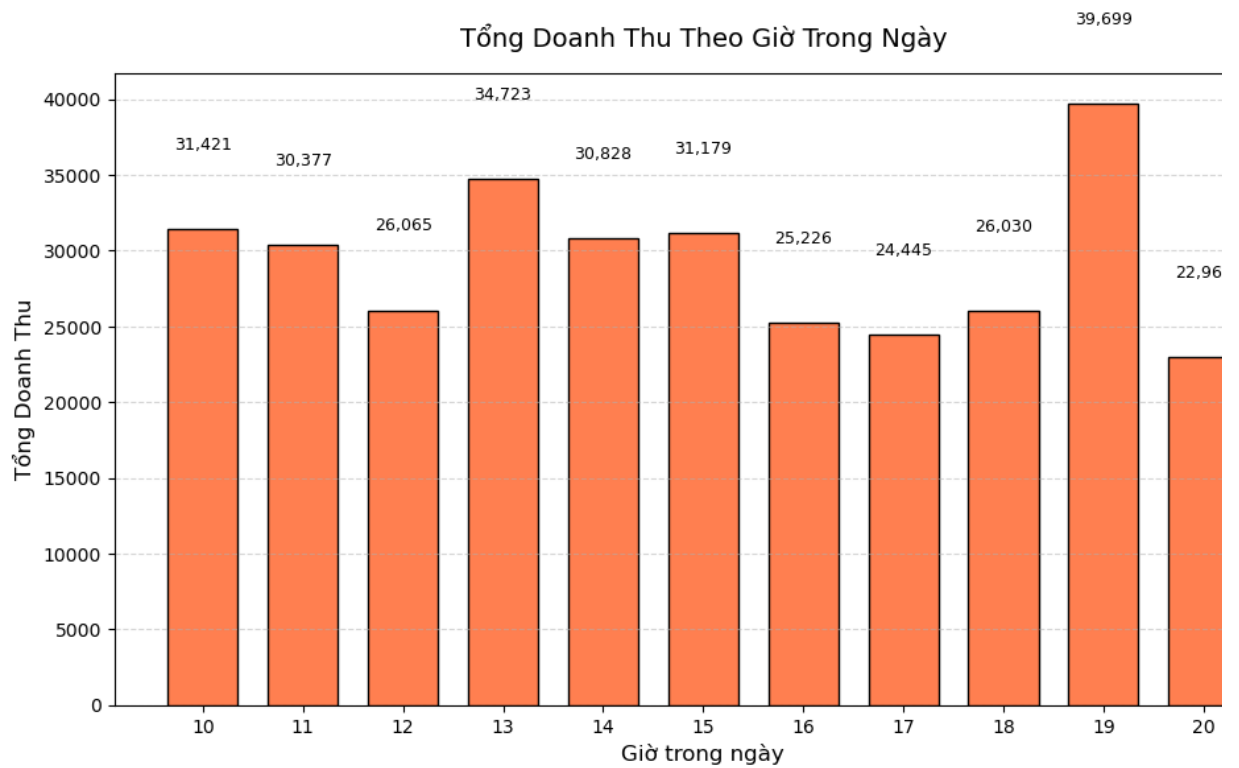
```

f'{int(yval):,}', ha='center', va='bottom', fontsize=9)

# Chỉ hiển thị ticks cho các giờ CÓ dữ liệu (không hiển thị 0-23 đầy đủ)
plt.xticks(hour_sales['Hour']) # <-- đây là thay đổi chính

plt.tight_layout()
plt.show()

```



```

In [48]: supermarket_data_copy['Day_of_week'] = supermarket_data_copy['Date'].dt.day_name()
supermarket_data_copy

```

Out[48]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Ta
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.
...	...	...	...	...	...	...	...	...	
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	1/1/
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	2/16/
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	12/31/
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	1/2/
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30

1000 rows × 26 columns

```
In [49]: supermarket_data_coppy['Is_weekend'] = supermarket_data_coppy['Date'].dt.weekday
# True = Saturday/Sunday, False = Monday-Friday
```

```
In [50]: # ===== Tạo cột thời gian (chạy 1 lần) =====

supermarket_data_coppy['Date'] = pd.to_datetime(supermarket_data_coppy['Date'])

# Tạo Hour (nếu chưa có)
if 'Hour' not in supermarket_data_coppy.columns:
    supermarket_data_coppy['Time'] = pd.to_datetime(supermarket_data_coppy['Time'])
    supermarket_data_coppy['Hour'] = supermarket_data_coppy['Time'].dt.hour

# Tạo Day_of_week (cách tốt nhất)
supermarket_data_coppy['Day_of_week'] = supermarket_data_coppy['Date'].dt.day_name()

# (Tùy chọn) Tạo thêm Is_weekend
supermarket_data_coppy['Is_weekend'] = supermarket_data_coppy['Date'].dt.weekday

# Kiểm tra nhanh
print(supermarket_data_coppy[['Date', 'Hour', 'Day_of_week', 'Is_weekend']].head())
```

	Date	Hour	Day_of_week	Is_weekend
0	2019-05-01	13	Wednesday	False
1	2019-08-03	10	Saturday	True
2	2019-03-03	13	Sunday	True
3	2019-01-27	20	Sunday	True
4	2019-08-02	10	Friday	False
5	2019-03-25	18	Monday	False
6	2019-02-25	14	Monday	False
7	2019-02-24	11	Sunday	True

```
In [51]: # Tạo bảng pivot: tổng doanh thu theo Ngày trong tuần và Giờ
heatmap_data = supermarket_data_coppy.pivot_table(
    values='Total',
    index='Day_of_week',
    columns='Hour',
    aggfunc='sum',
    fill_value=0
)

# Sắp xếp thứ tự ngày trong tuần cho đúng (từ Thứ 2 → Chủ Nhật)
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
heatmap_data = heatmap_data.reindex(day_order)

# Vẽ heatmap bằng matplotlib
fig, ax = plt.subplots(figsize=(14, 7)) # kích thước lớn để dễ nhìn

# Vẽ ma trận màu
im = ax.imshow(heatmap_data.values, cmap='YlOrRd', aspect='auto')

# Thêm colorbar (thanh màu bên phải)
cbar = fig.colorbar(im, ax=ax, pad=0.03)
cbar.set_label('Tổng Doanh Thu', rotation=270, labelpad=20, fontsize=11)

# Thiết lập nhãn trục x (giờ)
ax.set_xticks(range(len(heatmap_data.columns)))
ax.set_xticklabels(heatmap_data.columns, fontsize=10)

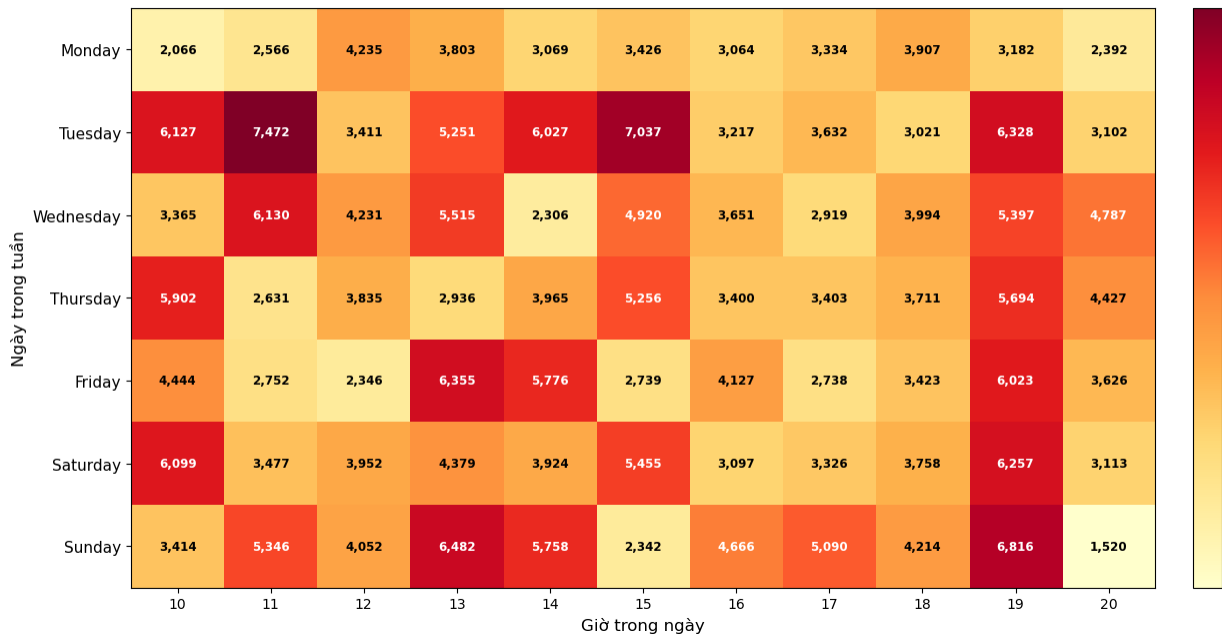
# Thiết lập nhãn trục y (ngày trong tuần)
ax.set_yticks(range(len(heatmap_data.index)))
ax.set_yticklabels(heatmap_data.index, fontsize=11)

# Thêm số doanh thu vào từng ô (chỉ ô có giá trị > 0 để tránh lộn xộn)
for i in range(len(heatmap_data.index)):
    for j in range(len(heatmap_data.columns)):
        value = heatmap_data.iloc[i, j]
        if value > 0:
            # Chọn màu chữ trắng nếu ô tối, đen nếu ô sáng
            text_color = 'white' if value > heatmap_data.values.max() * 0.6 else 'black'
            ax.text(j, i, f'{int(value):,}',
                    ha='center', va='center', color=text_color, fontsize=8.5, fontweight='bold')

# Tiêu đề và nhãn
ax.set_title('Heatmap Doanh Thu Theo Giờ Trong Ngày & Ngày Trong Tuần', fontsize=12)
ax.set_xlabel('Giờ trong ngày', fontsize=12)
ax.set_ylabel('Ngày trong tuần', fontsize=12)

# Cải thiện bố cục
plt.tight_layout()
plt.show()
```

Heatmap Doanh Thu Theo Giờ Trong Ngày & Ngày Trong Tuần



```
In [52]: # 2. Tạo DataFrame chỉ có doanh thu theo thứ
df_doanhthu_theo_thu = supermarket_data_coppy.groupby('Day_of_week')['Total'].sum().reset_index(name='Tổng Doanh Thu')

# 3. Sắp xếp đúng thứ tự: Thứ 2 → Chủ Nhật
thu_tu = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
df_doanhthu_theo_thu = df_doanhthu_theo_thu.set_index('Day_of_week').reindex(thu_tu)

# 4. Đổi tên cột tiếng Việt cho dễ đọc
df_doanhthu_theo_thu.columns = ['Thứ trong tuần', 'Tổng Doanh Thu']

# 5. Format số đẹp (thêm dấu phẩy, không thập phân)
df_doanhthu_theo_thu['Tổng Doanh Thu'] = df_doanhthu_theo_thu['Tổng Doanh Thu'].apply(lambda x: '{:,.0f}'.format(x))

# Hiển thị kết quả
print("=== DOANH THU THEO THỨ TRONG TUẦN ===")
print(df_doanhthu_theo_thu.to_string(index=False))
```

```
=== DOANH THU THEO THỨ TRONG TUẦN ===
```

```
Thứ trong tuần Tổng Doanh Thu
```

```
Monday 35,049
```

```
Tuesday 54,630
```

```
Wednesday 47,221
```

```
Thursday 45,166
```

```
Friday 44,353
```

```
Saturday 46,843
```

```
Sunday 49,705
```

Phân tích hành vi khách hàng theo nhóm

```
In [ ]:
```